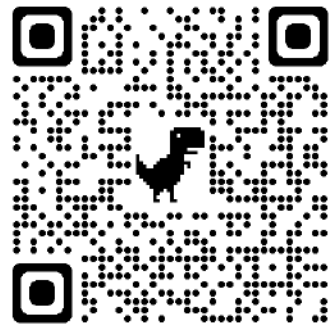


Lecture 13: Segmentation

Instructor: Roni Sengupta
ULA: Andrea Dunn, William Li,
Liujie Zheng



Course Website:
Scan Me!

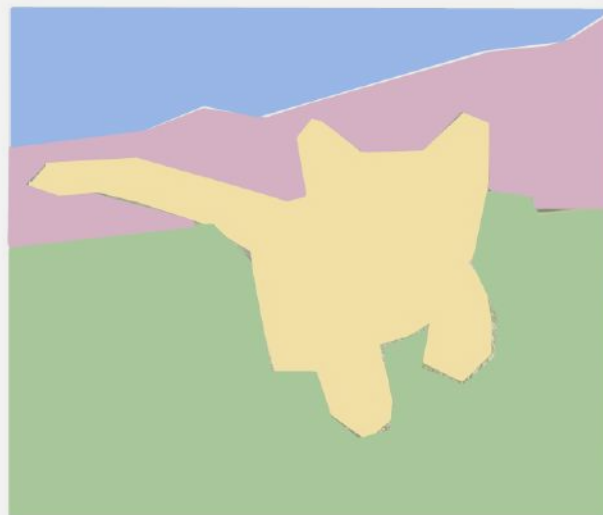
Classification



CAT

No spatial extent

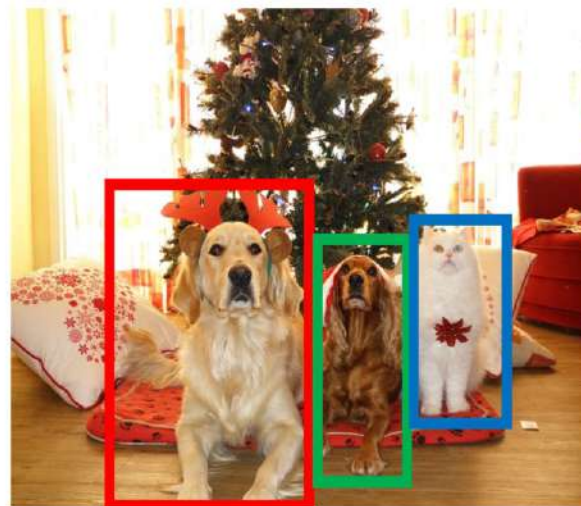
Semantic Segmentation



GRASS, CAT, TREE,
SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Objects

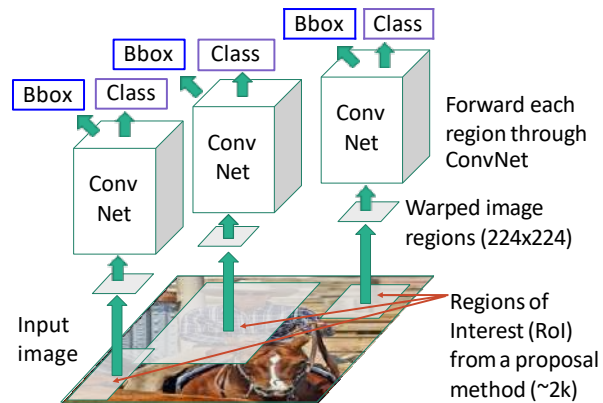
Instance Segmentation



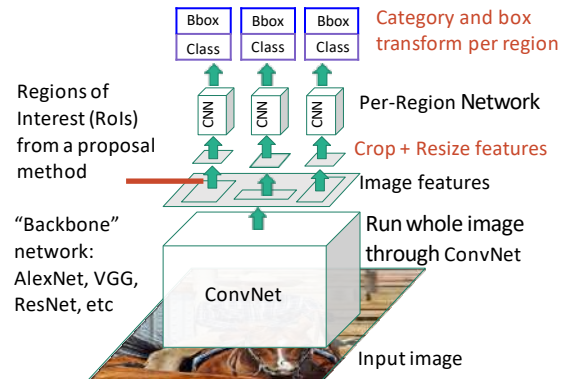
DOG, DOG, CAT

Recap

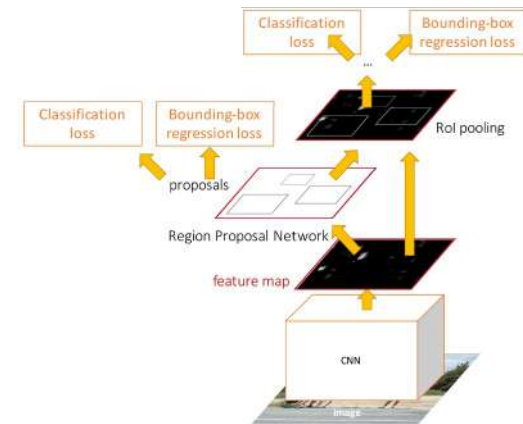
“Slow” R-CNN: Run CNN independently for each region



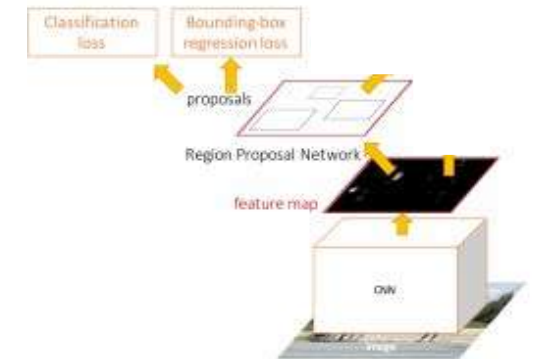
Fast R-CNN: Apply differentiable cropping to shared image features



Faster R-CNN: Compute proposals with CNN

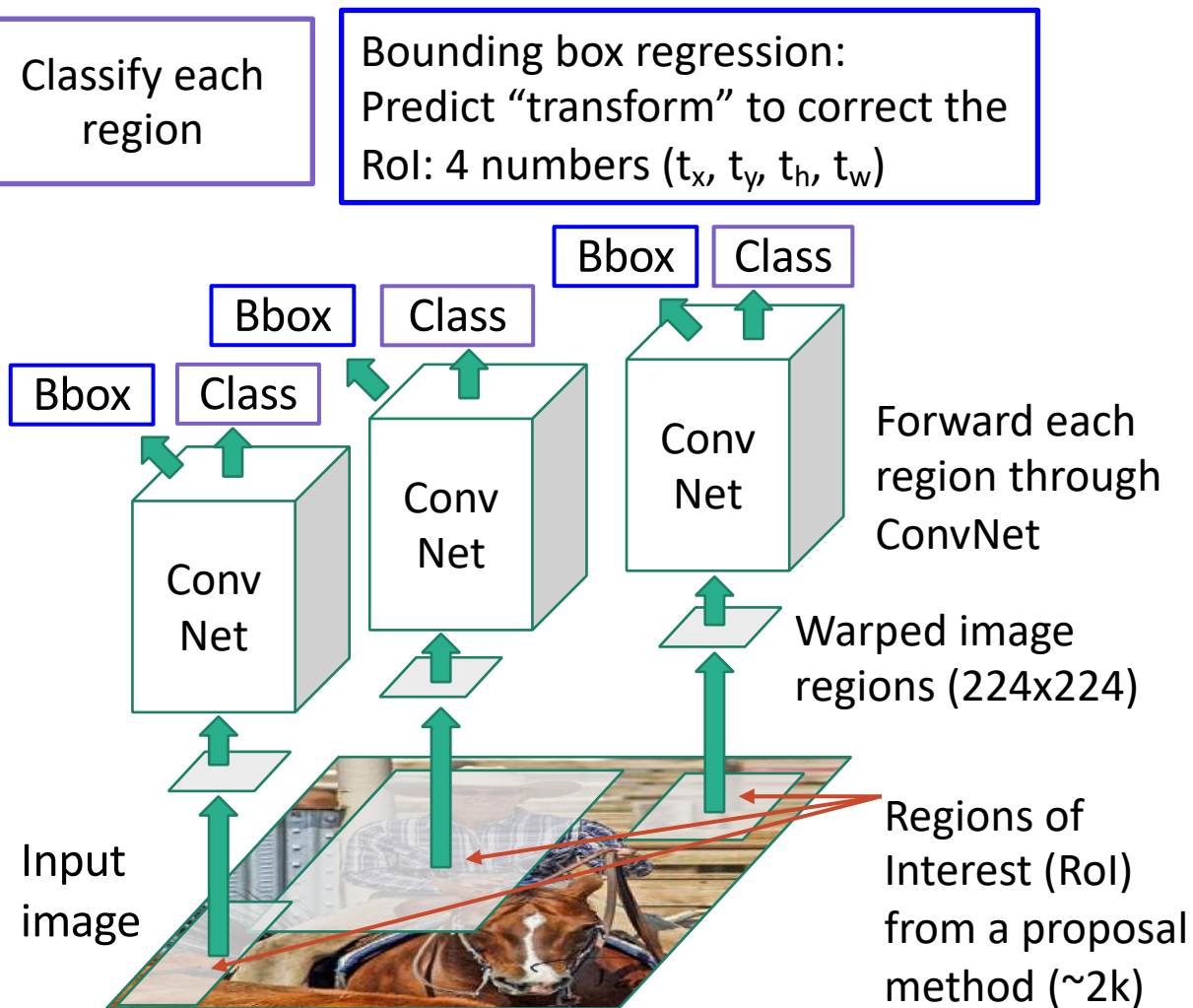


Single-Stage: Fully convolutional detector

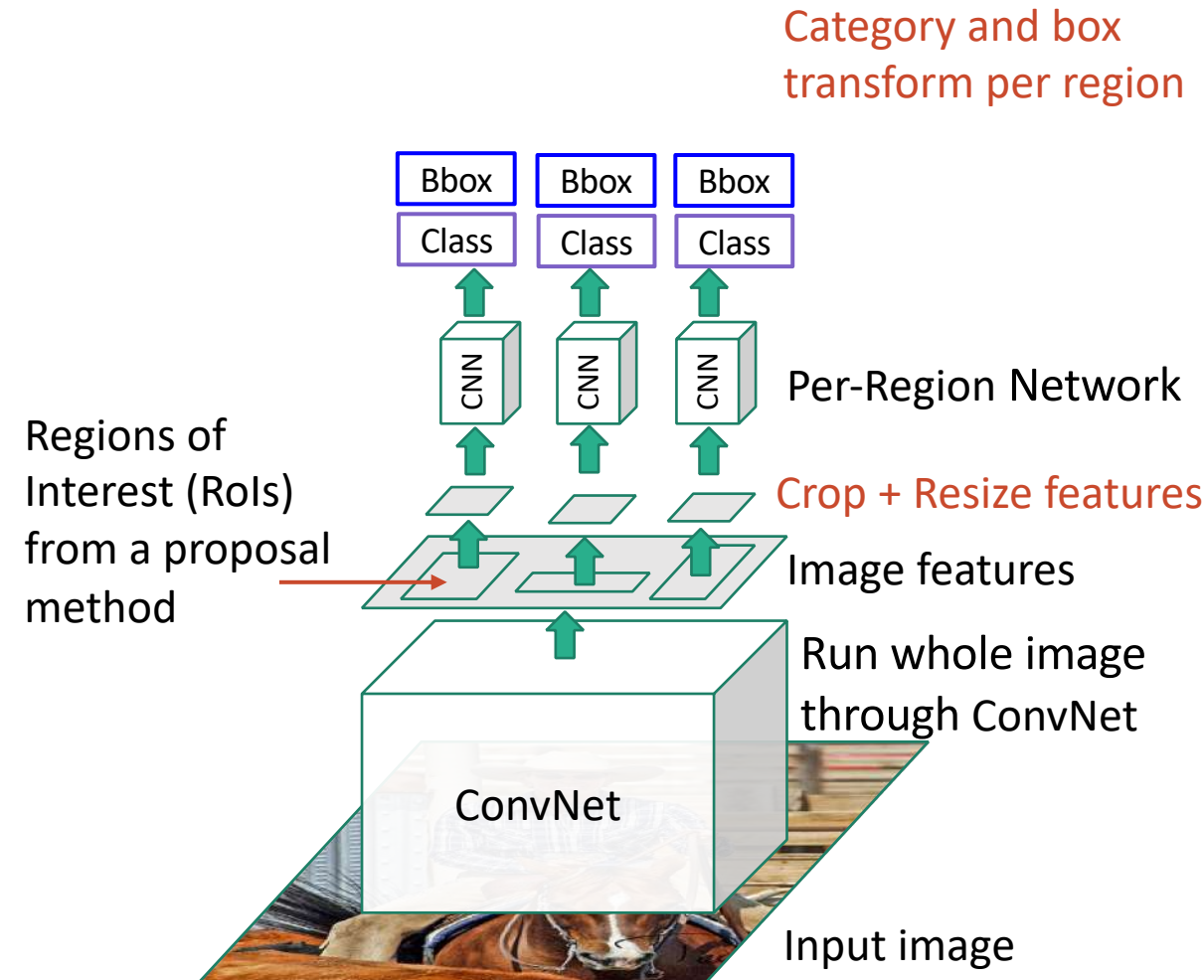


With anchors: RetinaNet
Anchor-Free: FCOS

R-CNN: Region-Based CNN



Fast R-CNN



Faster R-CNN: Learnable Region Proposals

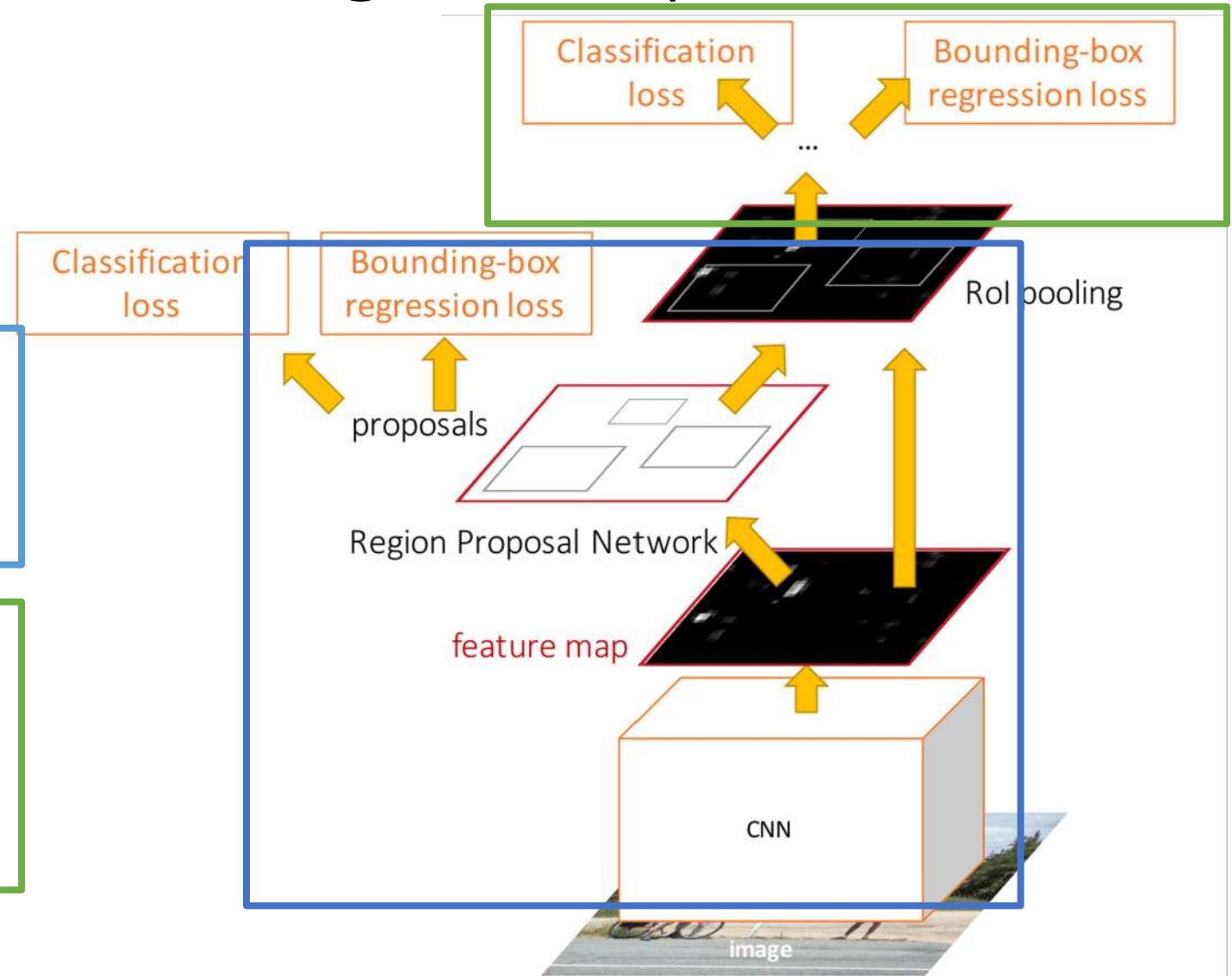
Faster R-CNN is a
Two-stage object detector

First stage: Run once per image

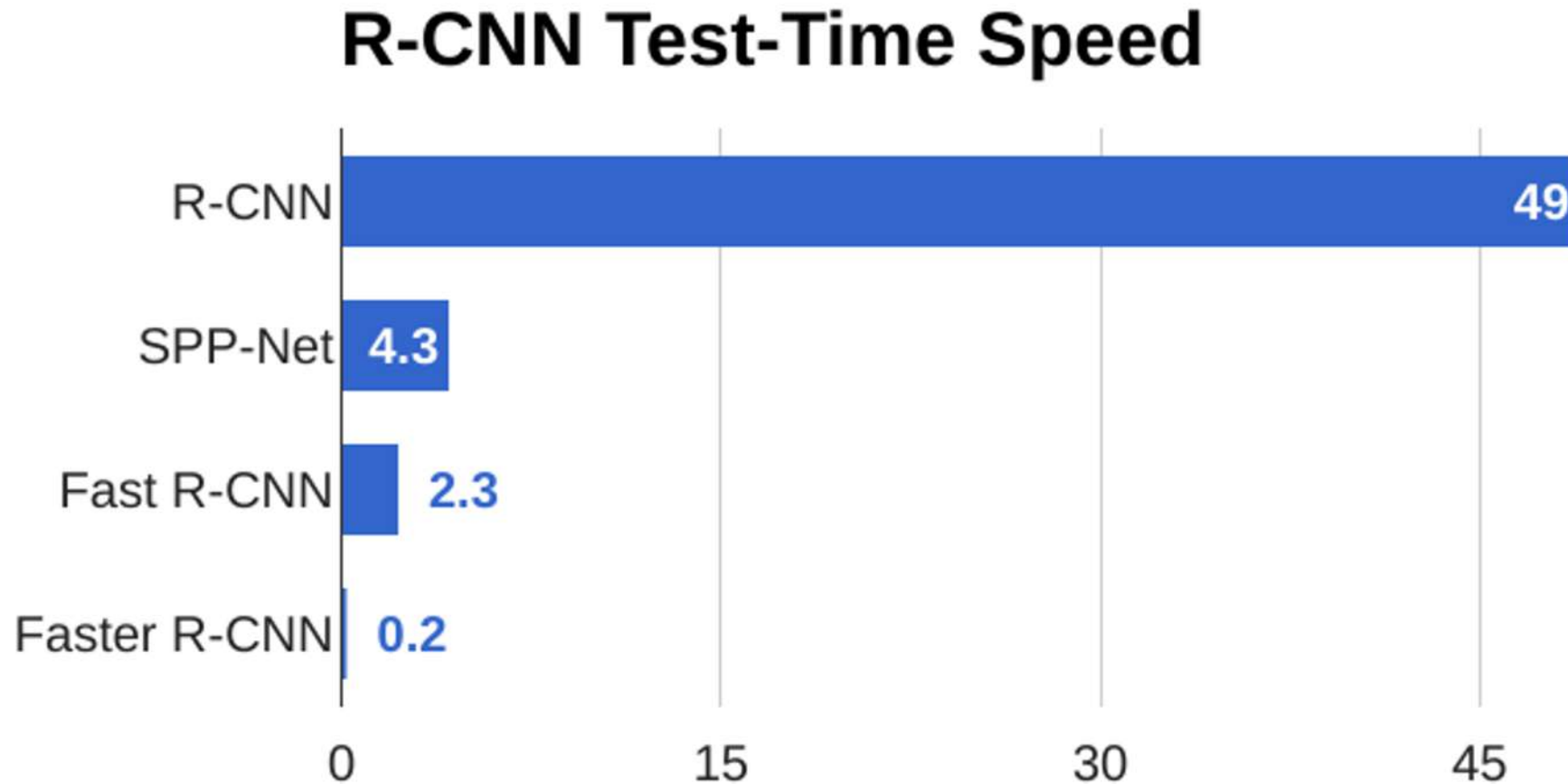
- Backbone network
- Region proposal network

Second stage: Run once per region

- Crop features: RoI pool / align
- Predict object class
- Prediction bbox offset

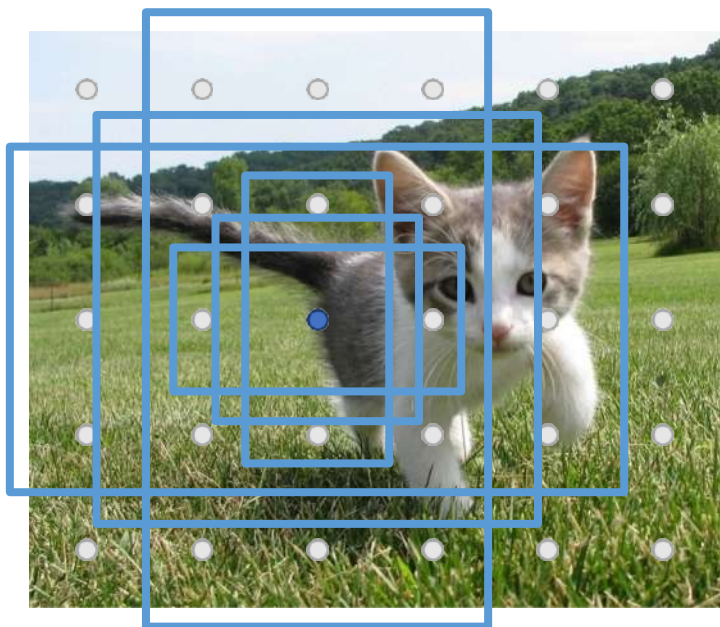


Faster R-CNN: Learnable Region Proposals



Single-Stage Detectors: RetinaNet

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)

Each feature corresponds to a point in the input

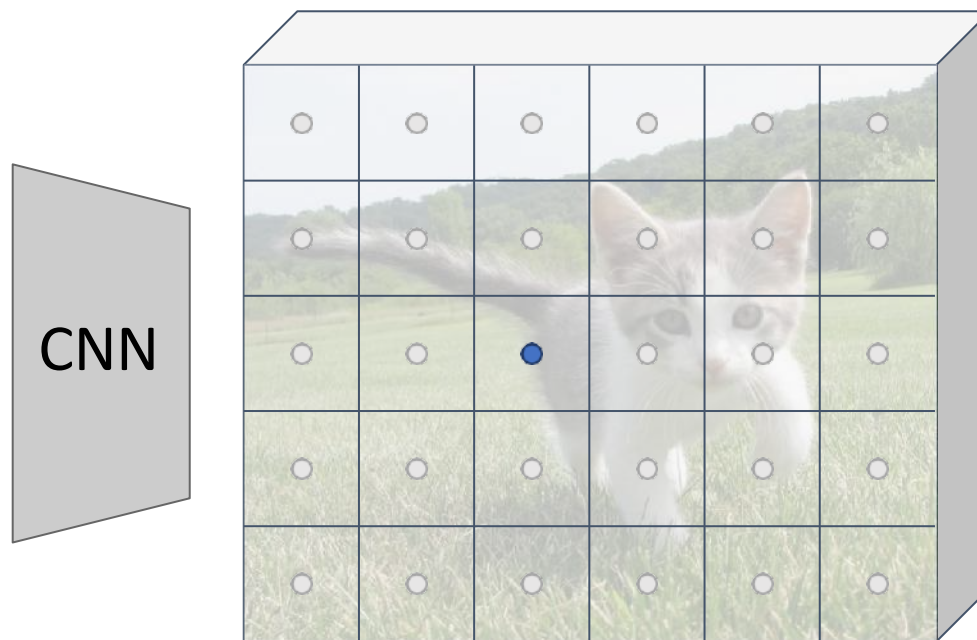
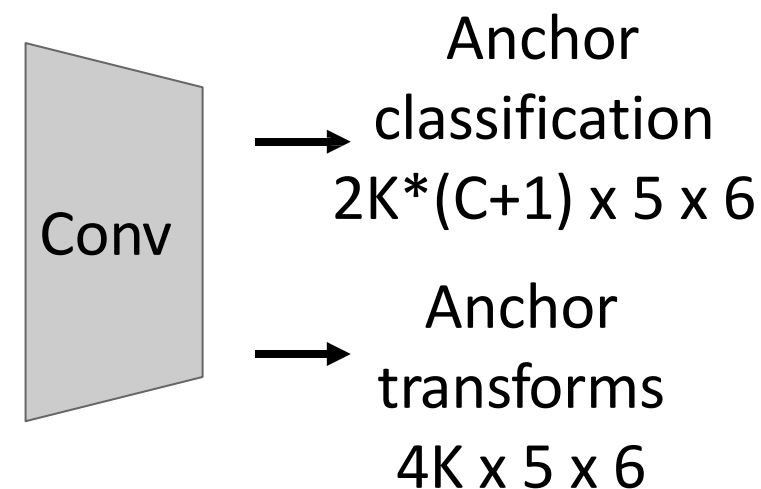


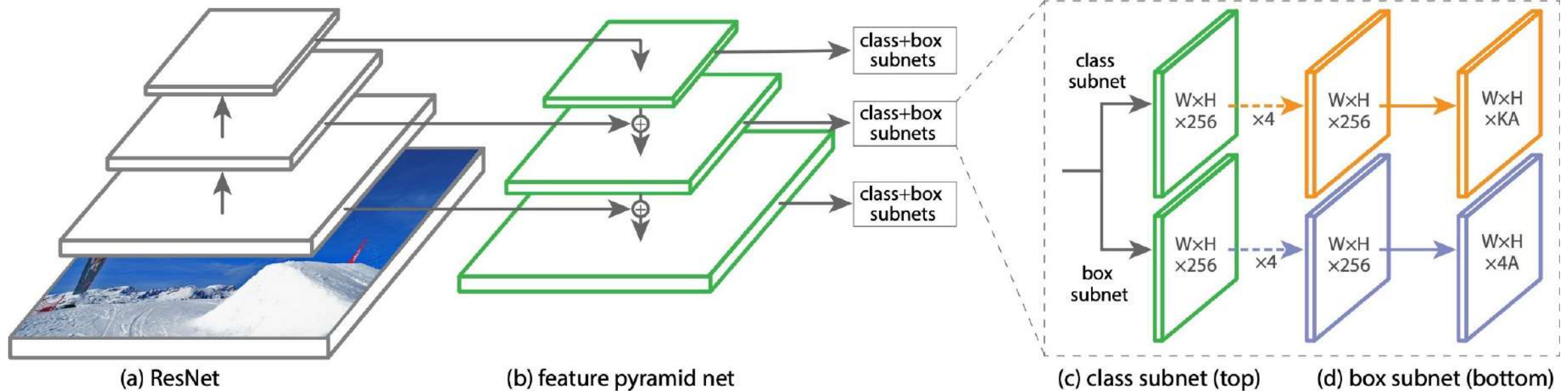
Image features
(e.g. 512 x 5 x 6)

Similar to RPN – but rather than classify anchors as object/no object, directly predict object category (among C categories) or background



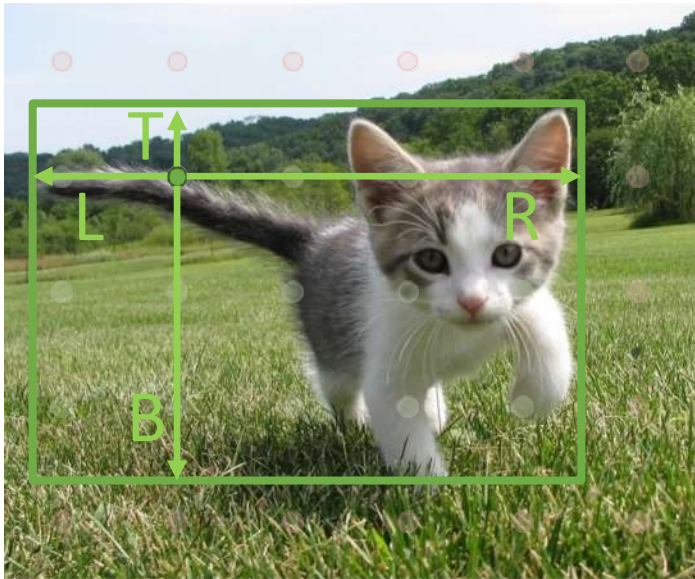
Single-Stage Detectors: RetinaNet

In practice, RetinaNet also uses Feature Pyramid Network to handle multiscale



Single-Stage Detectors: FCOS (“Anchor-free” detector)

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)

Each feature corresponds to a point in the input

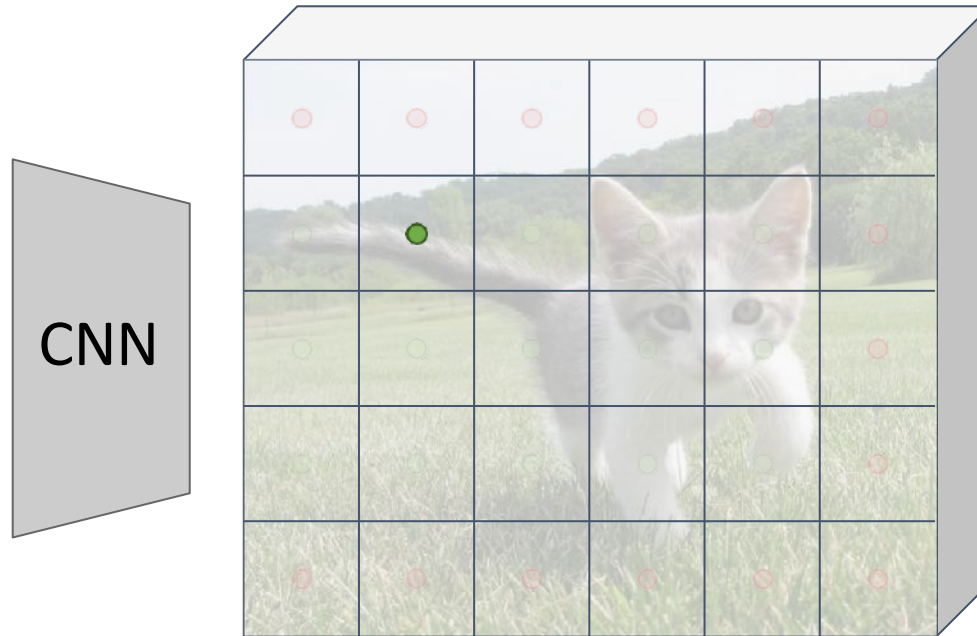
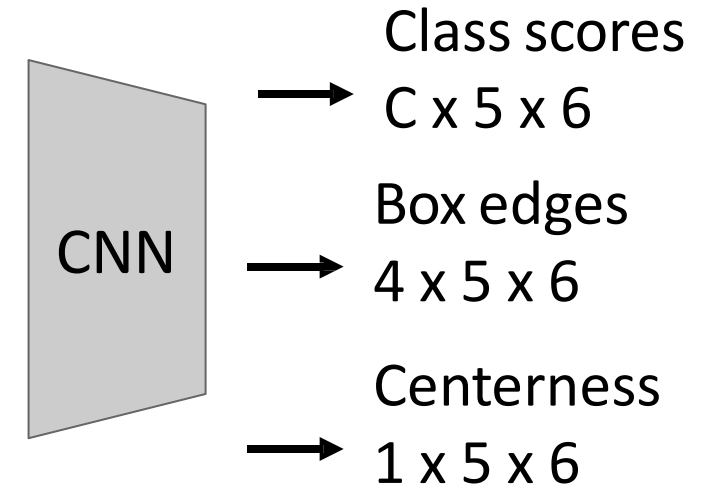


Image features
(e.g. 512 x 5 x 6)

Finally, predict “centerness” for all positive points (using logistic regression loss)



$$centerness = \sqrt{\frac{\min(L, R)}{\max(L, R)} \cdot \frac{\min(T, B)}{\max(T, B)}}$$

Ranges from 1 at box center to 0 at box edge

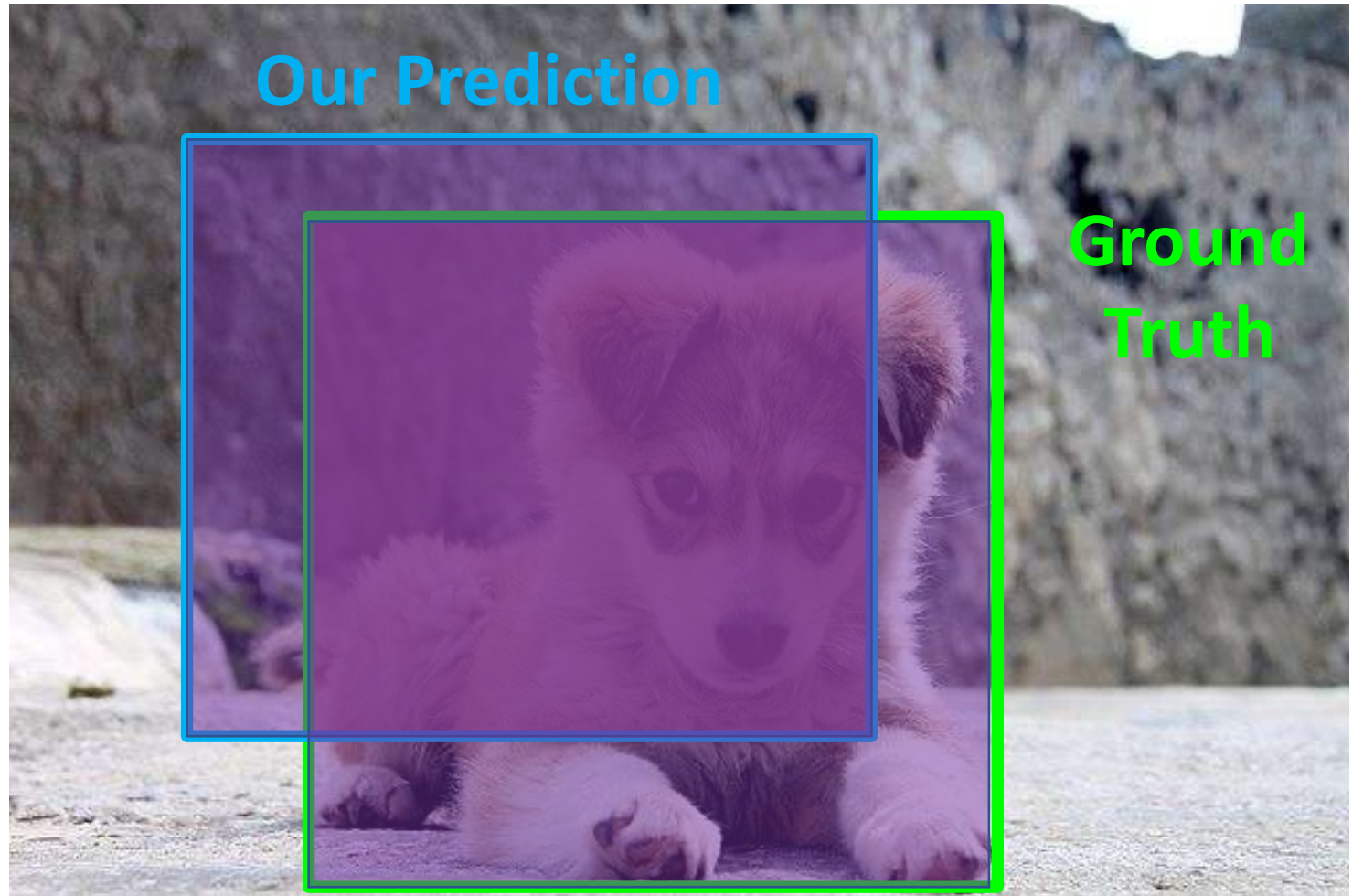
Test-time: predicted “confidence” for the box from each point is product of its class score and centerness.

Comparing Boxes: Intersection over Union (IoU)

How can we compare our prediction to the ground-truth box?

Intersection over Union (IoU)
(Also called “Jaccard similarity” or “Jaccard index”):

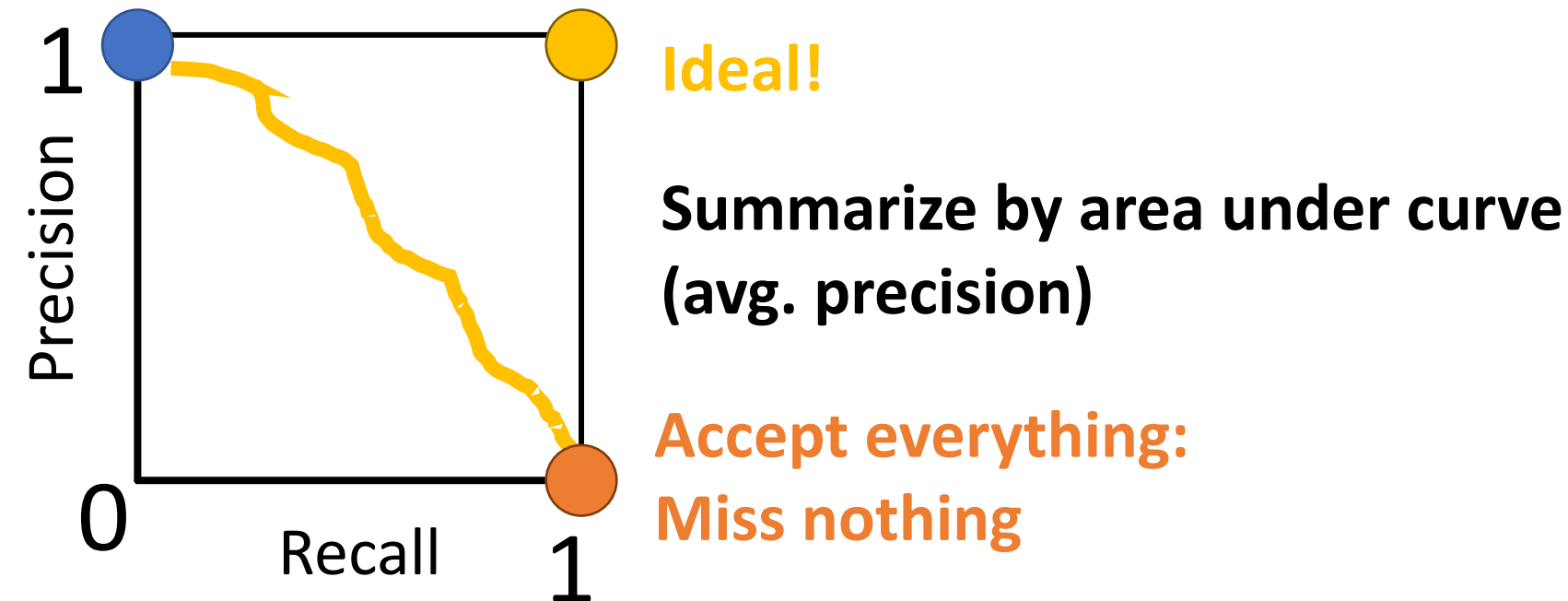
$$\frac{\text{Area of Intersection}}{\text{Area of Union}}$$



Precision & Recall

- True detection: high intersection over union
 - Choose IoU threshold
- Precision: $\# \text{true detections} / \# \text{detections}$
- Recall: $\# \text{true detections} / \# \text{true positives}$

Reject everything: no mistakes



Car AP = 0.65

Cat AP = 0.80

Dog AP = 0.86

mAP@0.5 = 0.77

mAP@0.5 = 0.77

mAP@0.55 = 0.71

mAP@0.60 = 0.65

...

mAP@0.95 = 0.2

COCO mAP = 0.4

Today's class

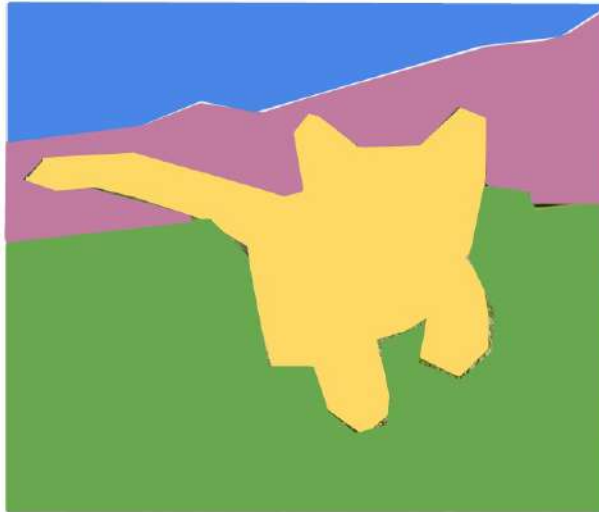
Classification



CAT

No spatial extent

Semantic Segmentation



GRASS, CAT, TREE,
SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Objects

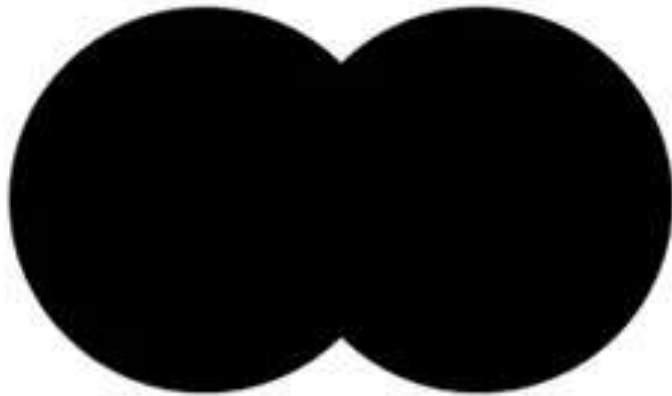
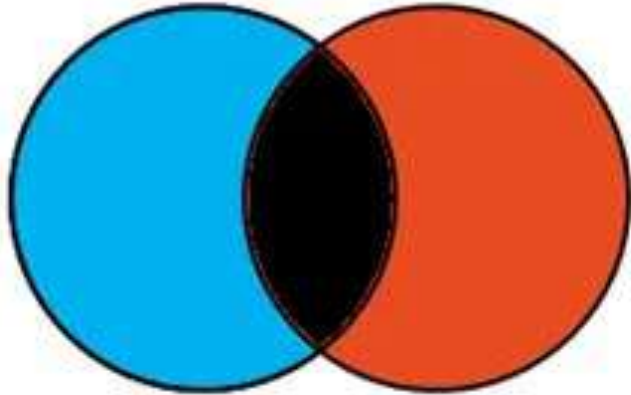
Instance Segmentation



DOG, DOG, CAT

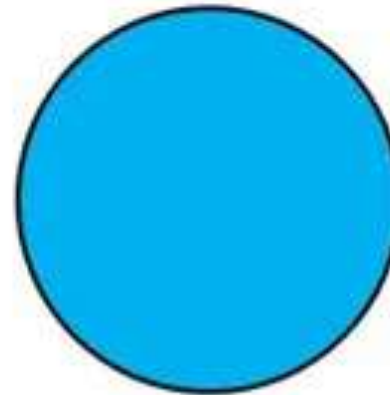
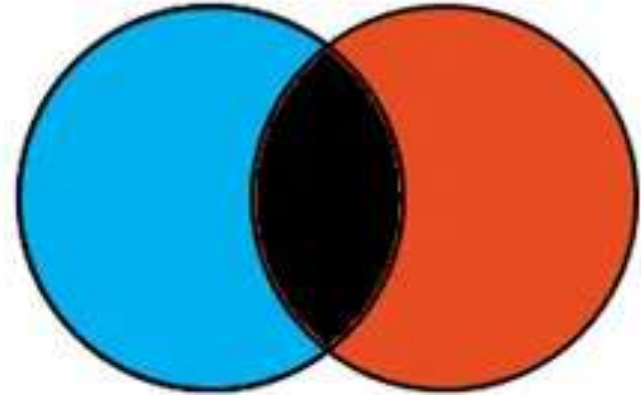
How to measure segmentation accuracy?

IoU (Intersection over Union)

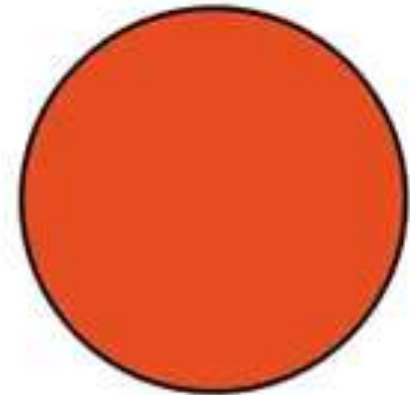


DICE Score (F score)

2 x



+



Report mean IoU or DICE score over the test dataset

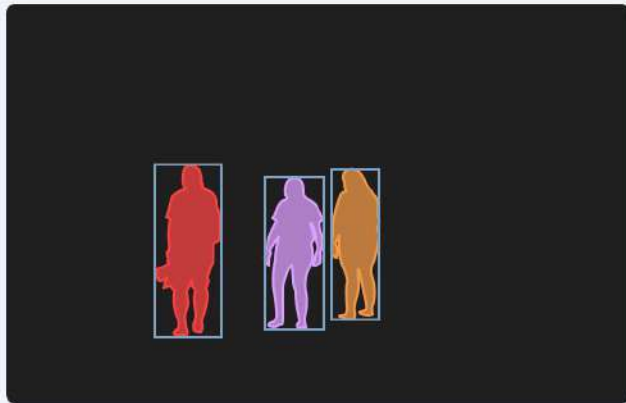
Semantic Segmentation vs. Instance Segmentation vs. Panoptic Segmentation



(a) Image



(b) Semantic Segmentation



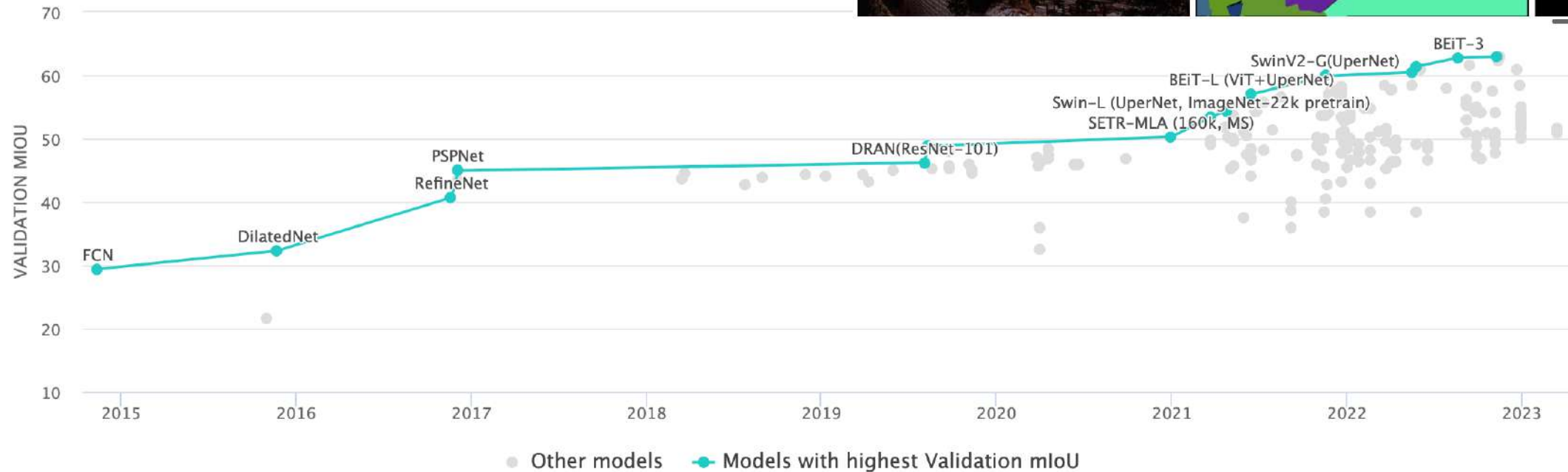
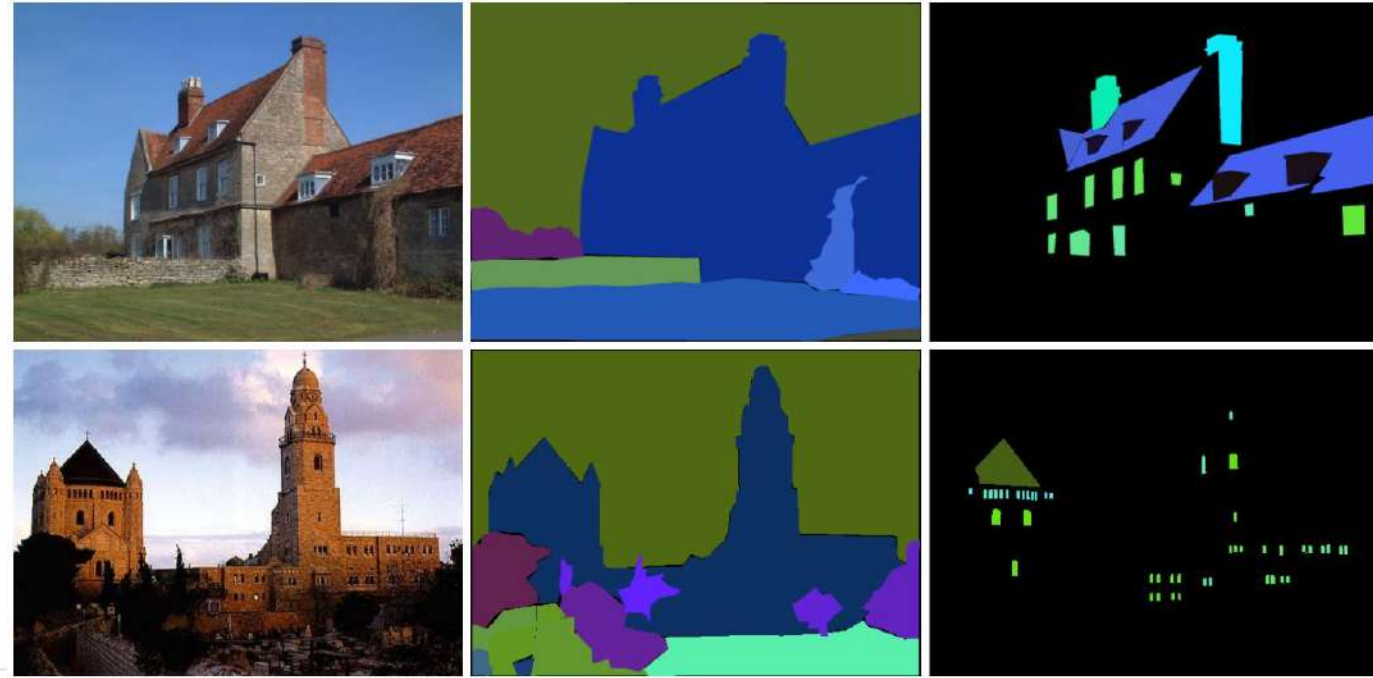
(c) Instance Segmentation



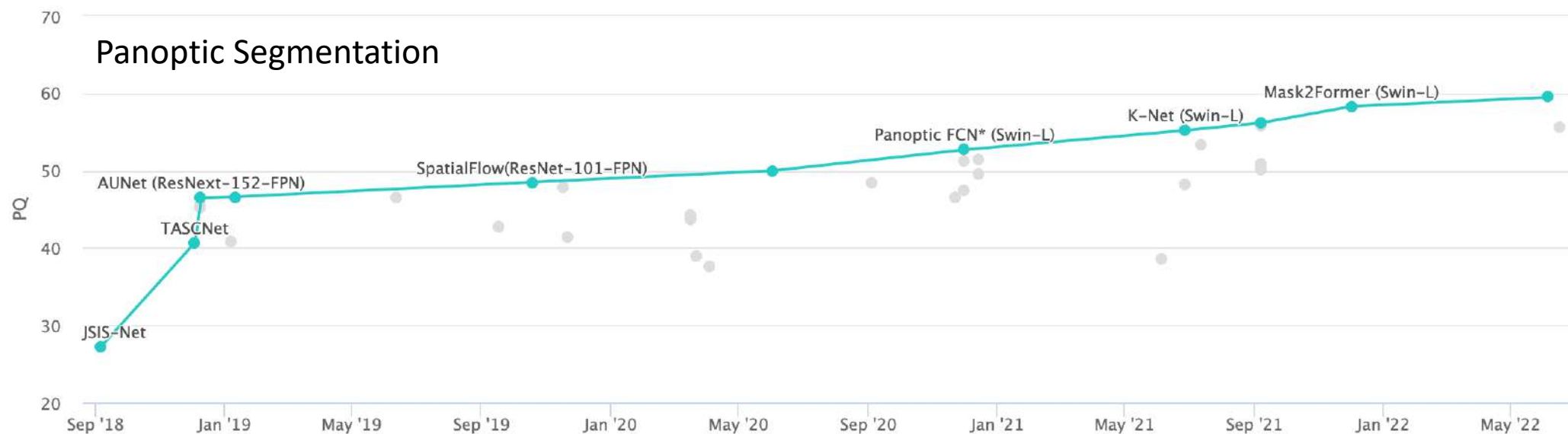
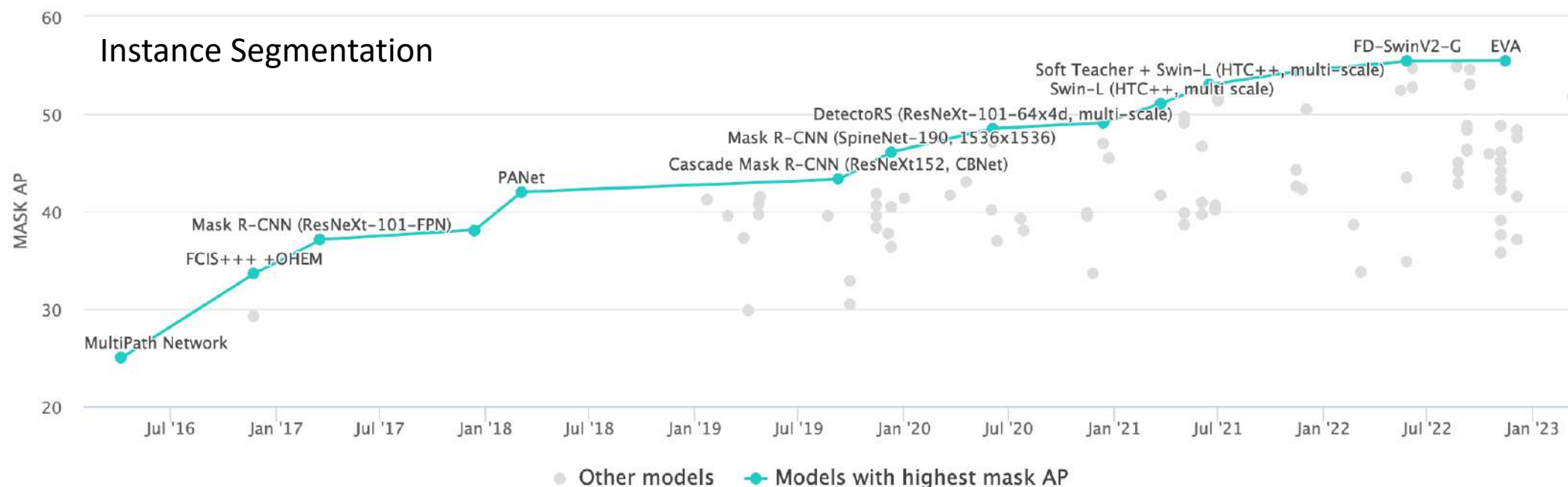
(d) Panoptic Segmentation

Most popular Semantic Segmentation dataset: ADE20k

- 20K scene-centric images exhaustively annotated with pixel-level objects and object parts labels.
- 150 semantic categories, which include stuffs like sky, road, grass, and discrete objects like person, car, bed.



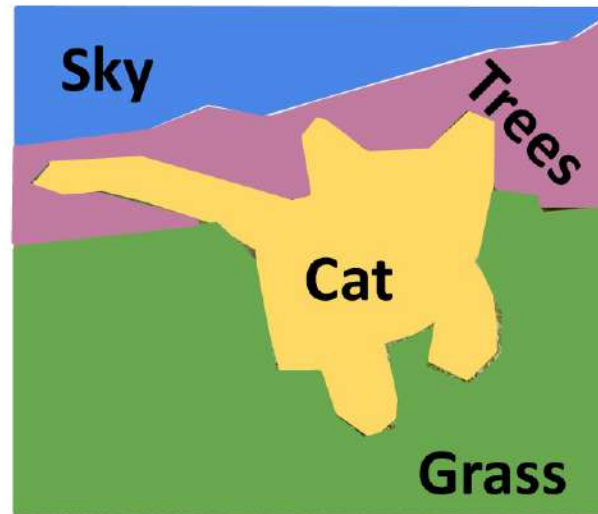
Most popular Instance & Panoptic Segmentation dataset: MS COCO



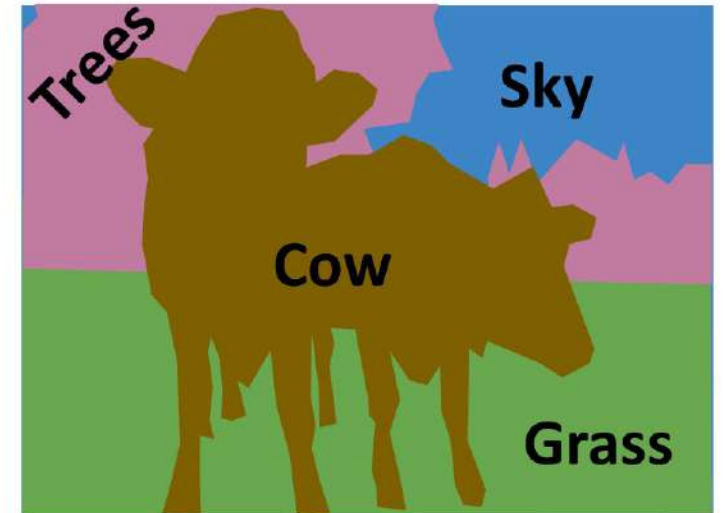
Semantic Segmentation

Label each pixel in the image with a category label

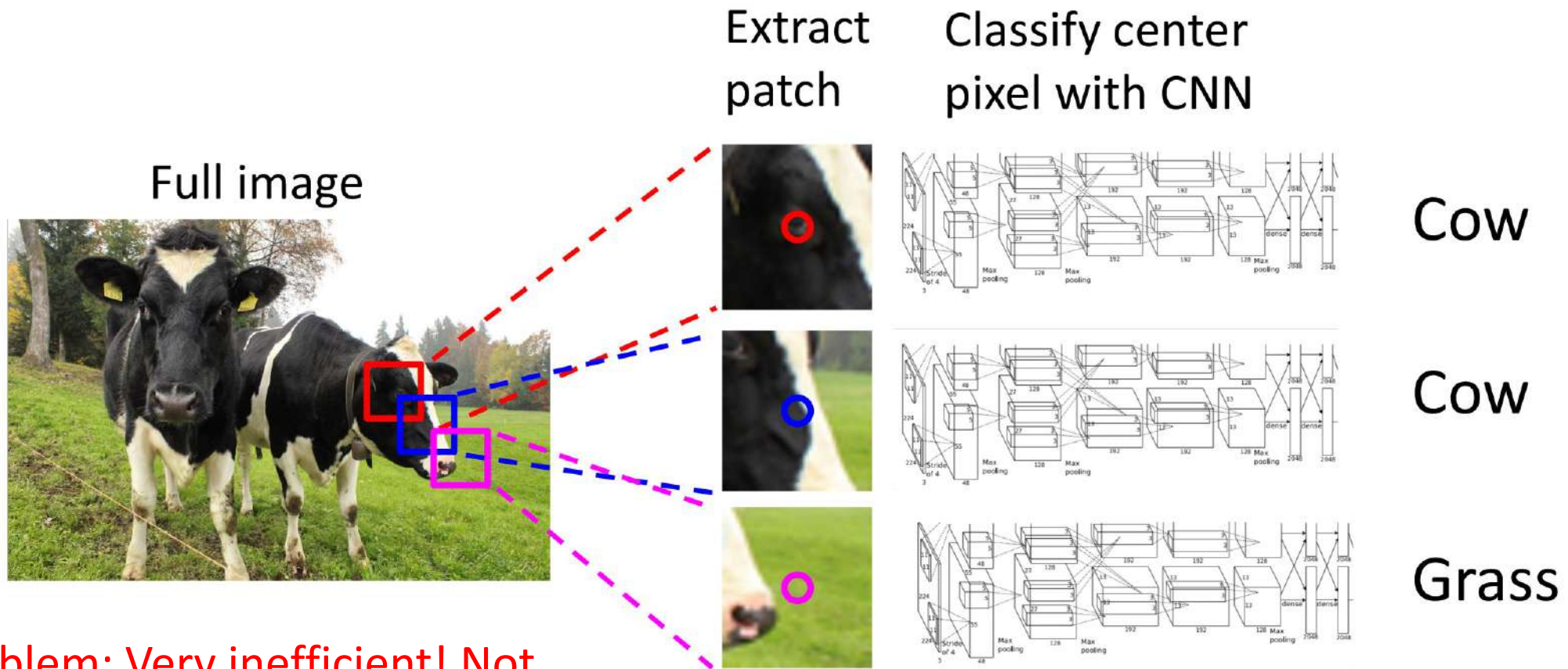
Don't differentiate instances, only care about pixels



[This image is CC0 public domain](#)



Semantic Segmentation Idea: Sliding Window

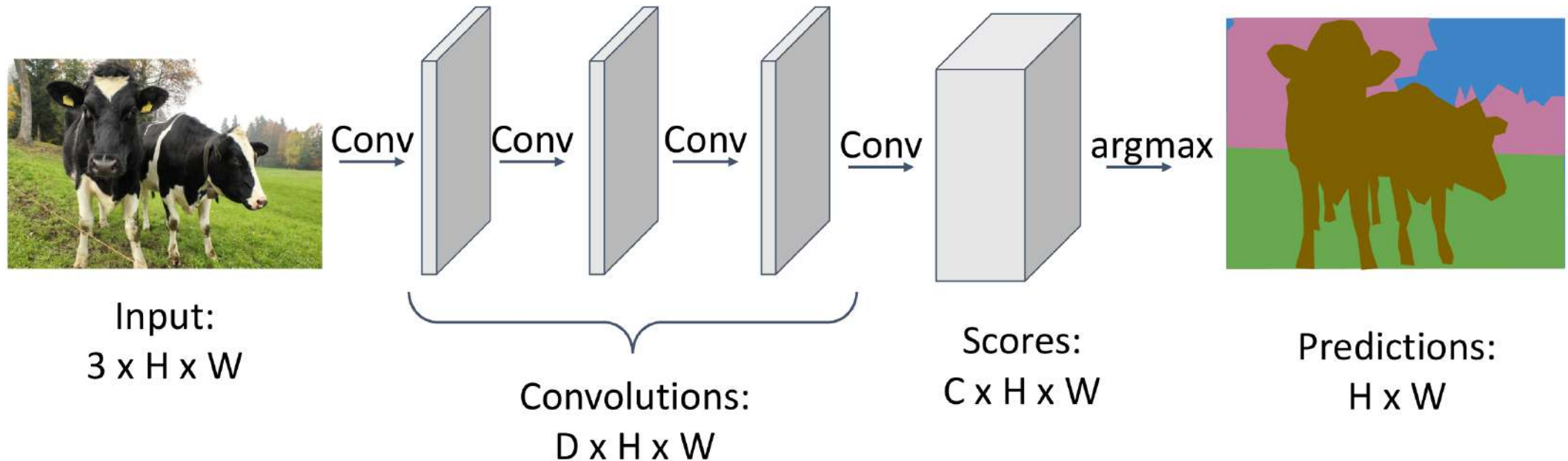


Problem: Very inefficient! Not reusing shared features between overlapping patches

Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013
Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

Semantic Segmentation: Fully Convolutional Network

Design a network as a bunch of convolutional layers to make predictions for pixels all at once!



Long et al, "Fully convolutional networks for semantic segmentation", CVPR 2015

Loss function: Per-Pixel cross-entropy

Problem #1: Effective receptive field size is linear in number of conv layers: With L 3×3 conv layers, receptive field is $1+2L$

Problem #2: Convolution on high res images is expensive! Recall ResNet stem aggressively downsamples

Semantic Segmentation: Fully Convolutional Network

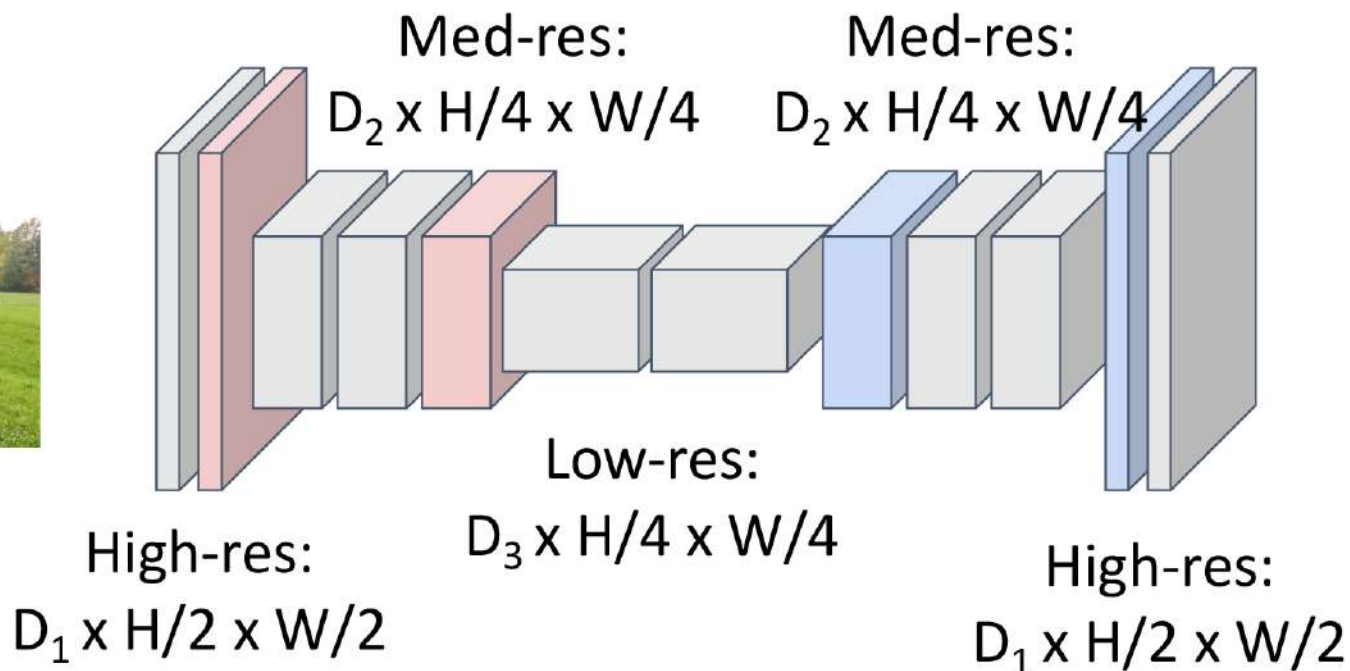
Downsampling:
Pooling, strided
convolution

Design network as a bunch of convolutional layers, with
downsampling and **upsampling** inside the network!

Upsampling: ???



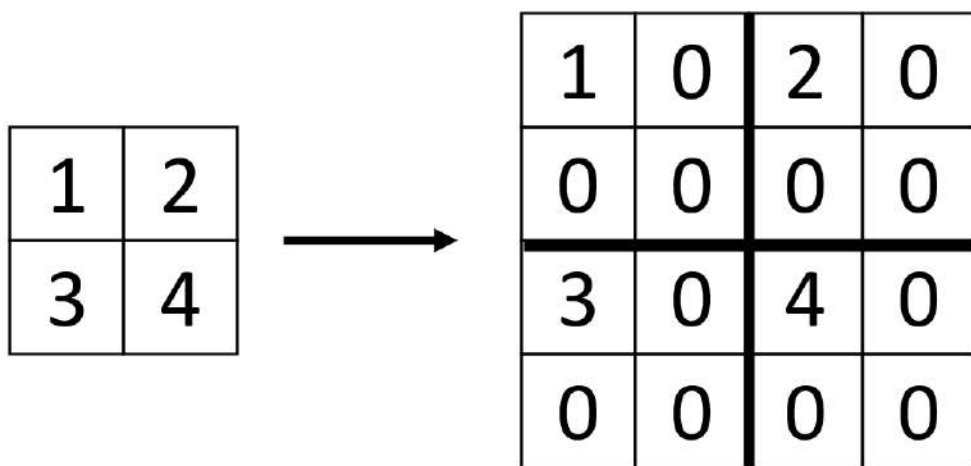
Input:
 $3 \times H \times W$



Predictions:
 $H \times W$

In-Network Upsampling: “Unpooling”

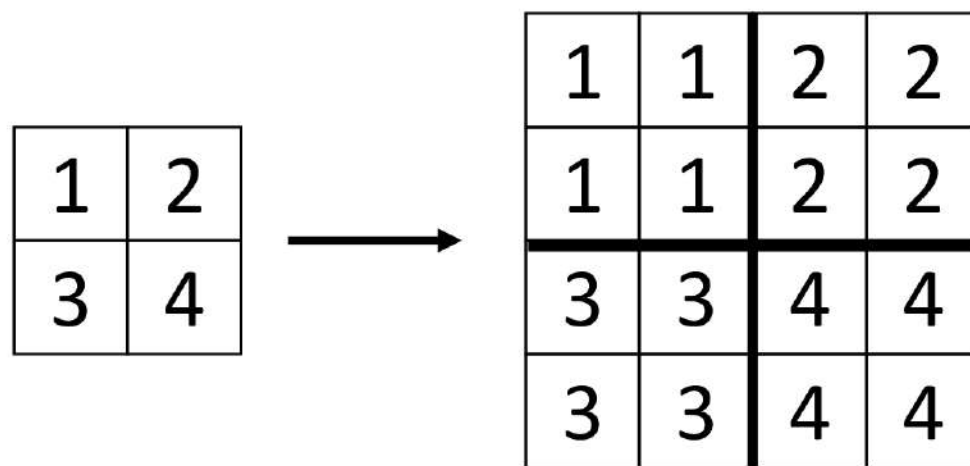
Bed of Nails



Input
 $C \times 2 \times 2$

Output
 $C \times 4 \times 4$

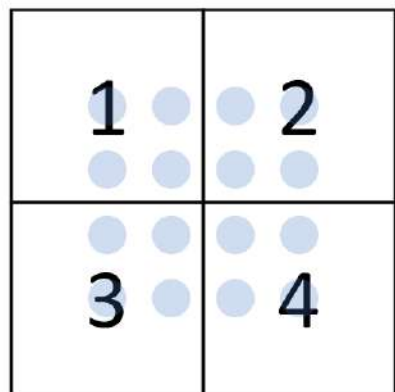
Nearest Neighbor



Input
 $C \times 2 \times 2$

Output
 $C \times 4 \times 4$

In-Network Upsampling: Bilinear Interpolation



Input: C x 2 x 2

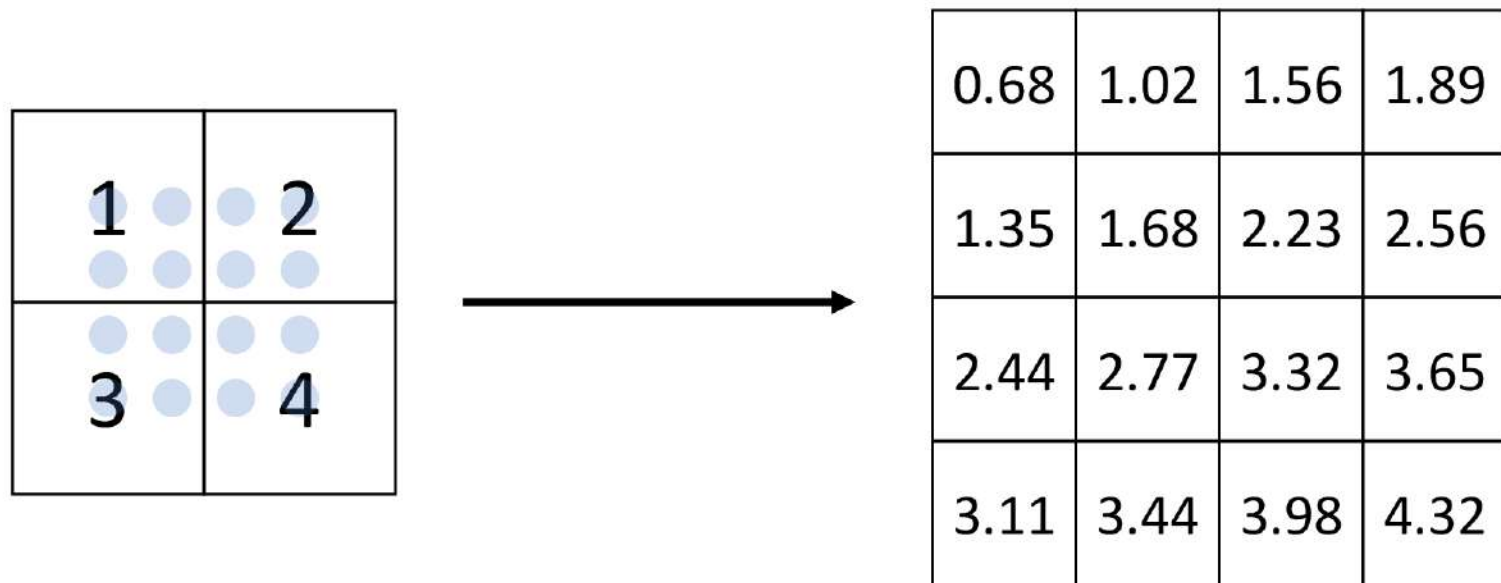
1.00	1.25	1.75	2.00
1.50	1.75	2.25	2.50
2.50	2.75	3.25	3.50
3.00	3.25	3.75	4.00

Output: C x 4 x 4

$$f_{x,y} = \sum_{i,j} f_{i,j} \max(0, 1 - |x - i|) \max(0, 1 - |y - j|) \quad \begin{array}{l} i \in \{\lfloor x \rfloor - 1, \dots, \lceil x \rceil + 1\} \\ j \in \{\lfloor y \rfloor - 1, \dots, \lceil y \rceil + 1\} \end{array}$$

Use two closest neighbors in x and y to construct linear approximations

In-Network Upsampling: Bicubic Interpolation



Input: $C \times 2 \times 2$

Output: $C \times 4 \times 4$

Use **three** closest neighbors in x and y to
construct **cubic** approximations
(This is how we normally resize images!)

In-Network Upsampling: “Max Unpooling”

Max Pooling: Remember which position had the max

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8



5	6
7	8



Rest
of
net

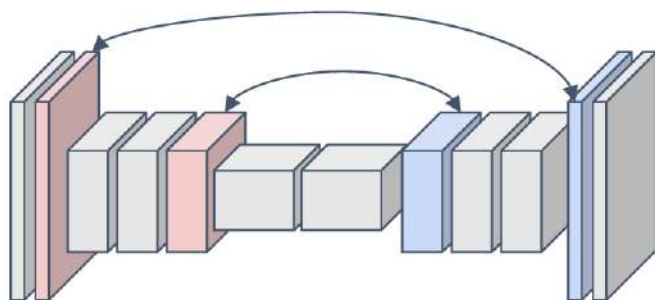


1	2
3	4



0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

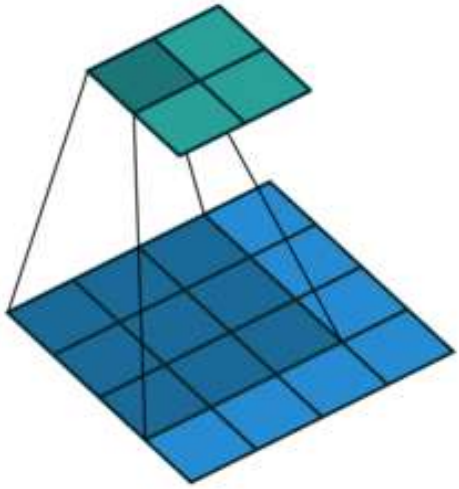
Max Unpooling: Place into remembered positions



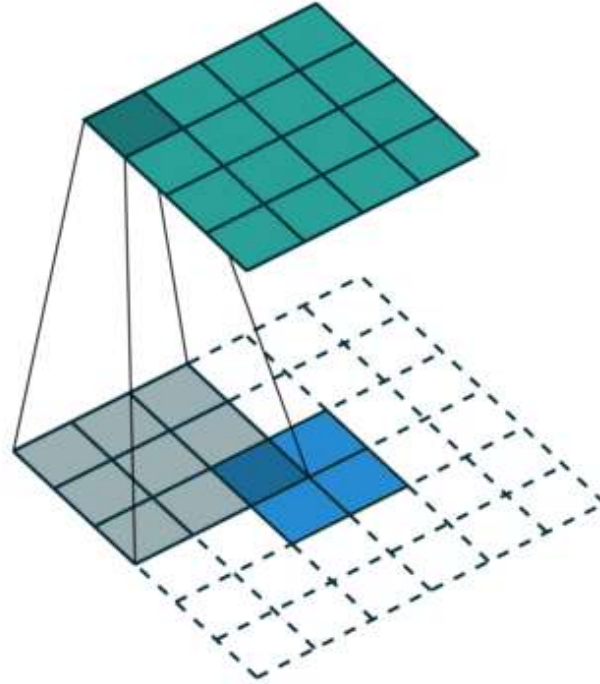
Pair each downsampling layer
with an upsampling layer

Regular vs Transposed Convolution

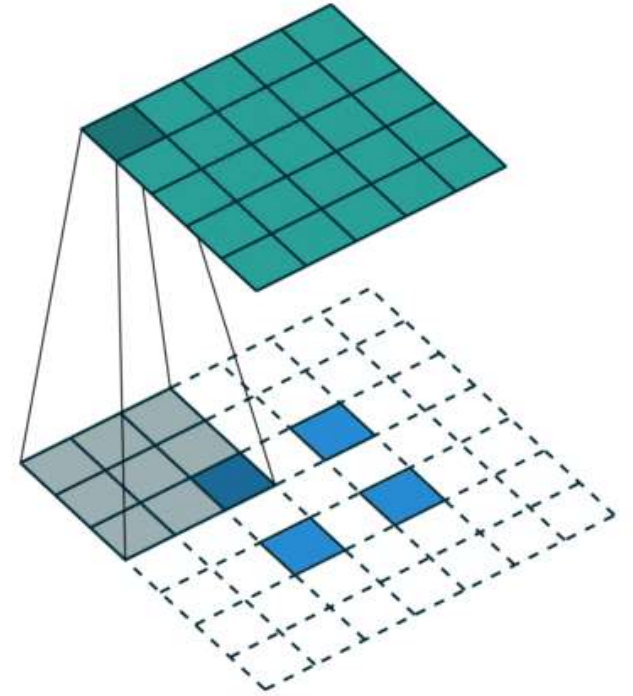
Filter size is 3x3



Regular Convolution
reduces feature size



Transposed convolution increases feature size



Strided transpose convolution

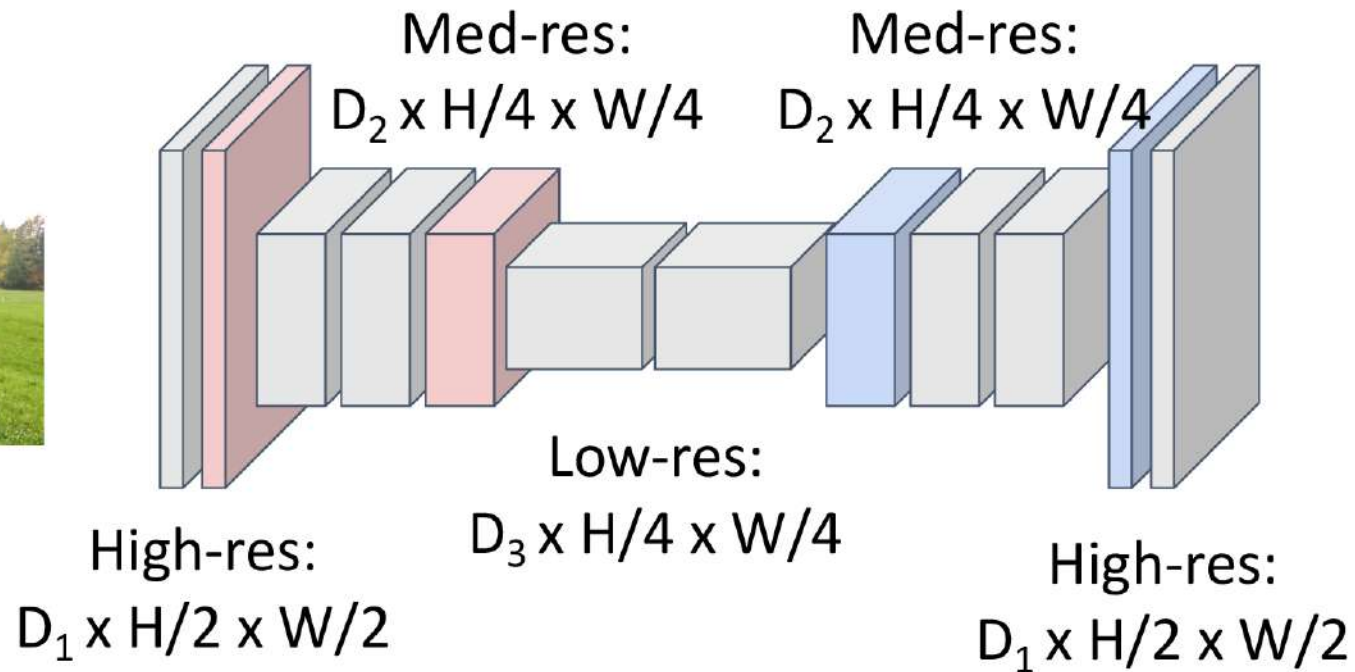
Semantic Segmentation: Fully Convolutional Network

Downsampling:
Pooling, strided
convolution

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Input:
 $3 \times H \times W$



Upsampling: ???



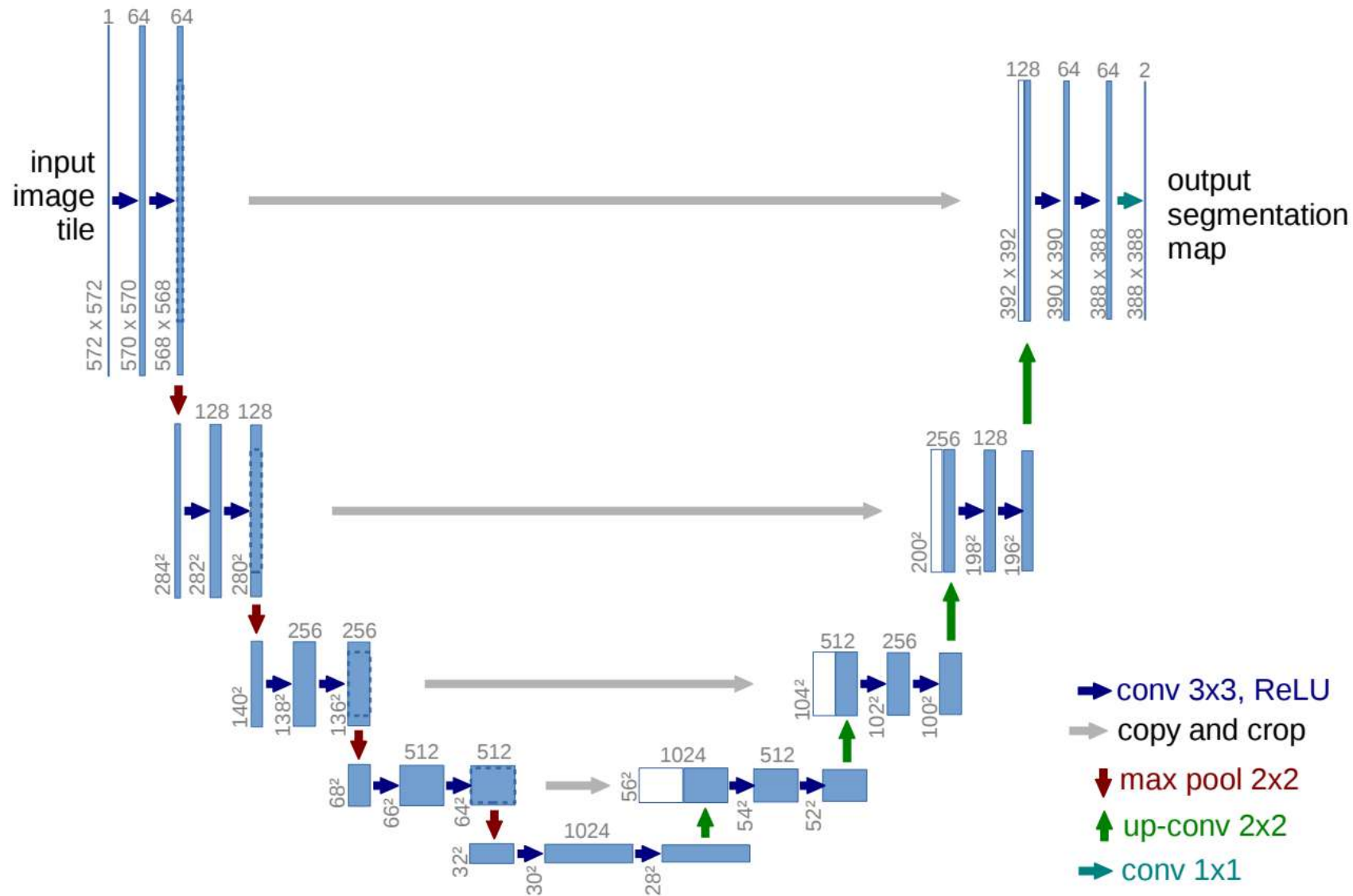
Predictions:
 $H \times W$

Upsampling:

Bilinear Upsampling (non learnable)

Strided Transpose Convolution (learnable)

U-Net for segmentation



SOTA Detection + Segmentation

Swin Transformer

 Ranked #13	Object Detection on COCO test-dev	 Ranked #6	Instance Segmentation on COCO test-dev (using additional training data)
 Ranked #10	Semantic Segmentation on ADE20K (using additional training data)		
 Ranked #24	Action Classification on Kinetics-400 (using additional training data)		

This repo is the official implementation of "[Swin Transformer: Hierarchical Vision Transformer using Shifted Windows](#)" as well as the follow-ups. It currently includes code and models for the following tasks:

Image Classification: Included in this repo. See [get_started.md](#) for a quick start.

Object Detection and Instance Segmentation: See [Swin Transformer for Object Detection](#).

Semantic Segmentation: See [Swin Transformer for Semantic Segmentation](#).

Video Action Recognition: See [Video Swin Transformer](#).

Semi-Supervised Object Detection: See [Soft Teacher](#).

SSL: Contrastive Learning: See [Transformer-SSL](#).

SSL: Masked Image Modeling: See [get_started.md#simmim-support](#).

Mixture-of-Experts: See [get_started](#) for more instructions.

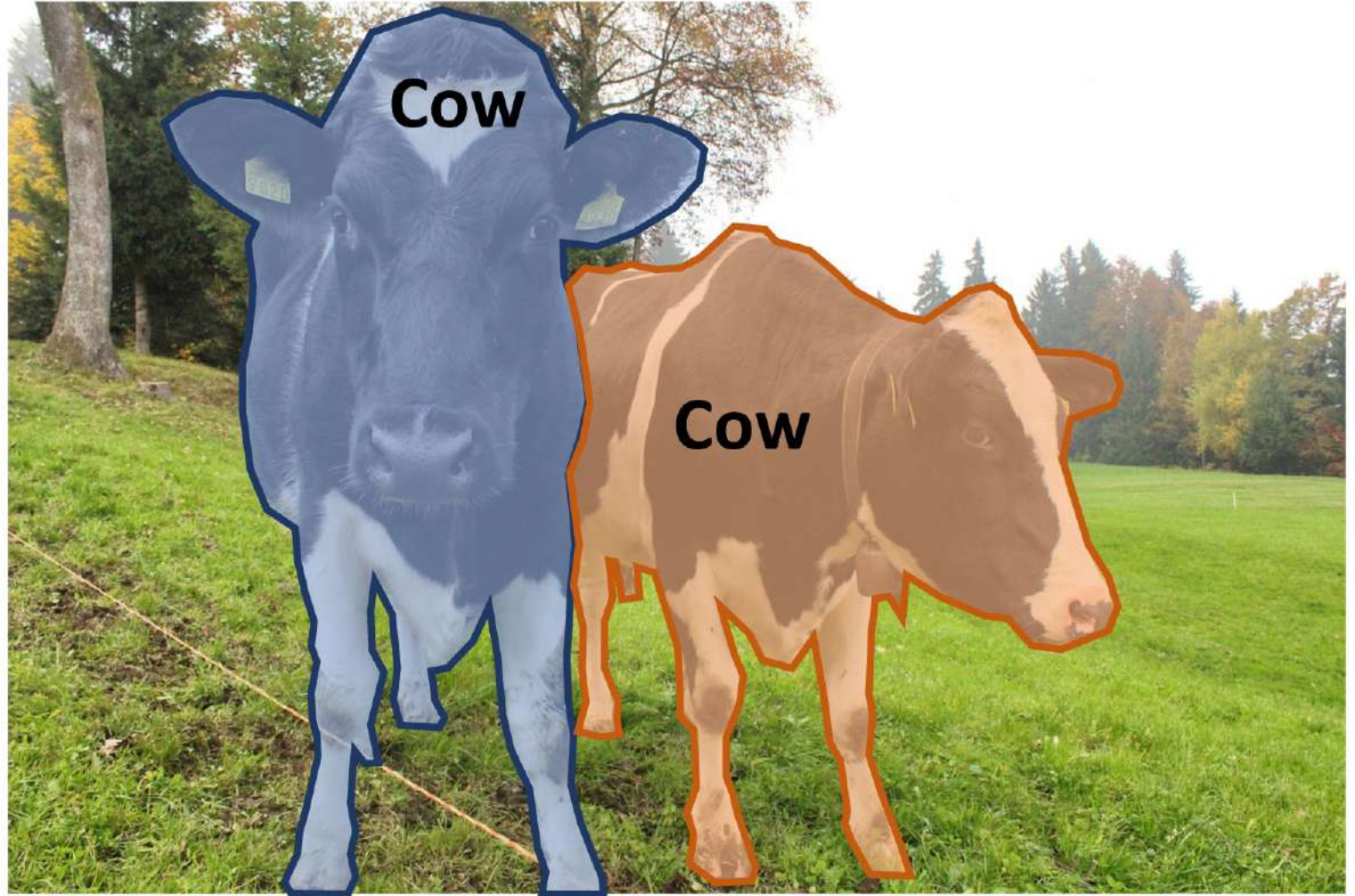
Feature-Distillation: See [Feature-Distillation](#).

Computer Vision Tasks: Instance Segmentation

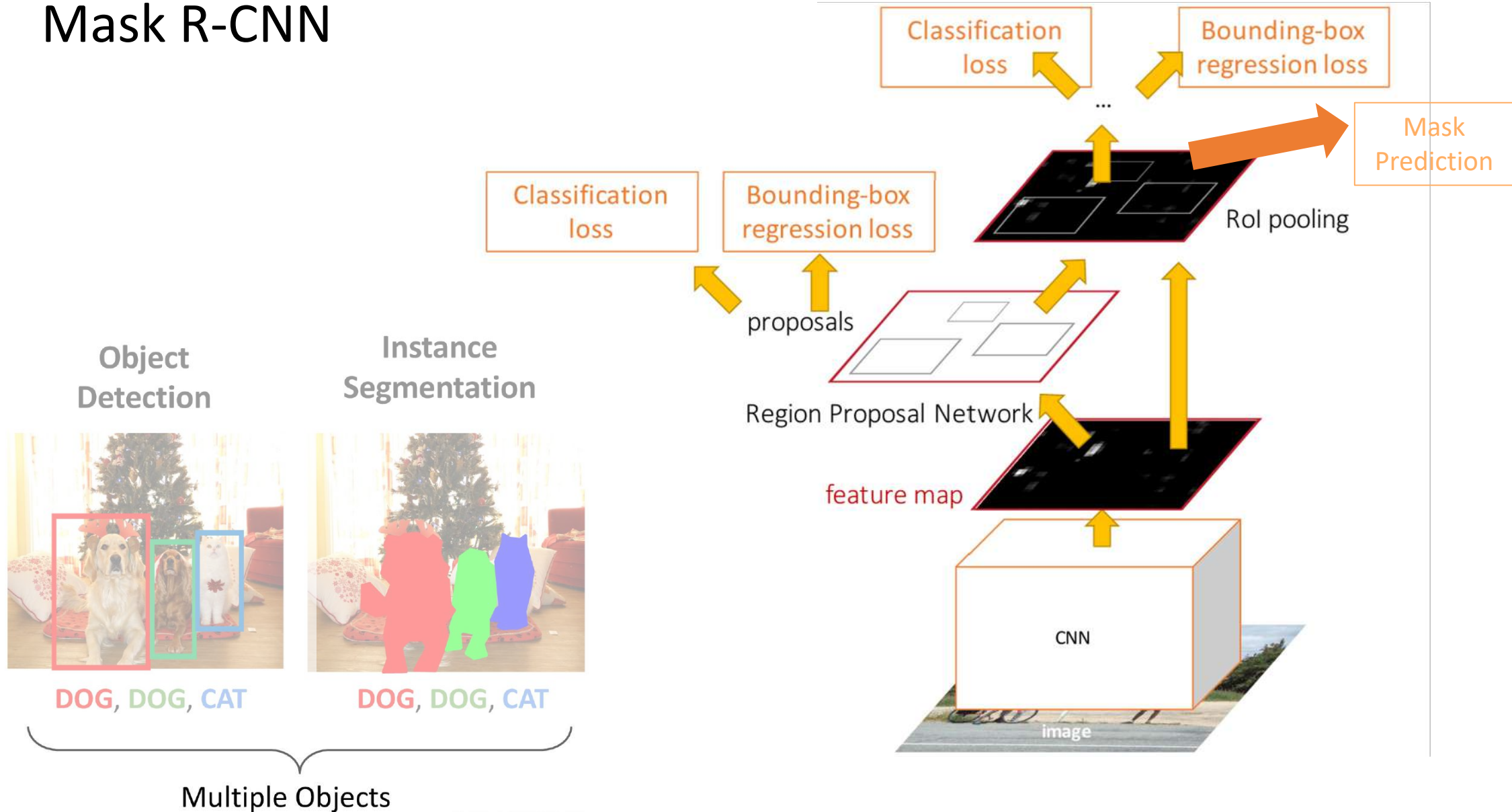
Instance Segmentation:

Detect all objects in the image, and identify the pixels that belong to each object (Only things!)

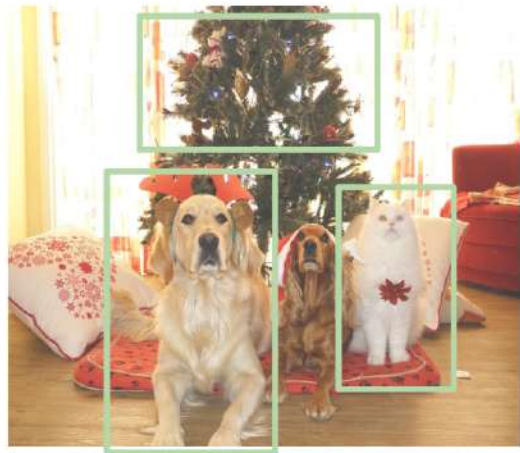
Approach: Perform object detection, then predict a segmentation mask for each object!



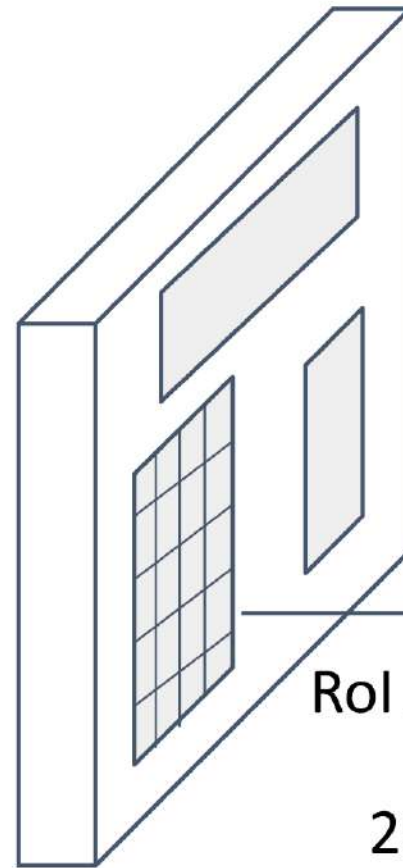
Instance segmentation with Mask R-CNN



Mask R-CNN



CNN
+RPN



RoI Align

256 x 14 x 14

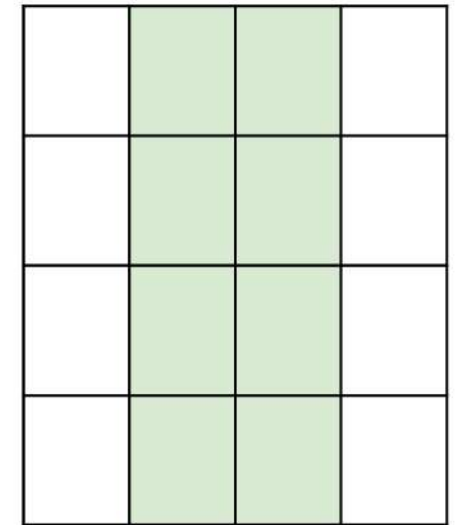


Conv

256 x 14 x 14



Conv

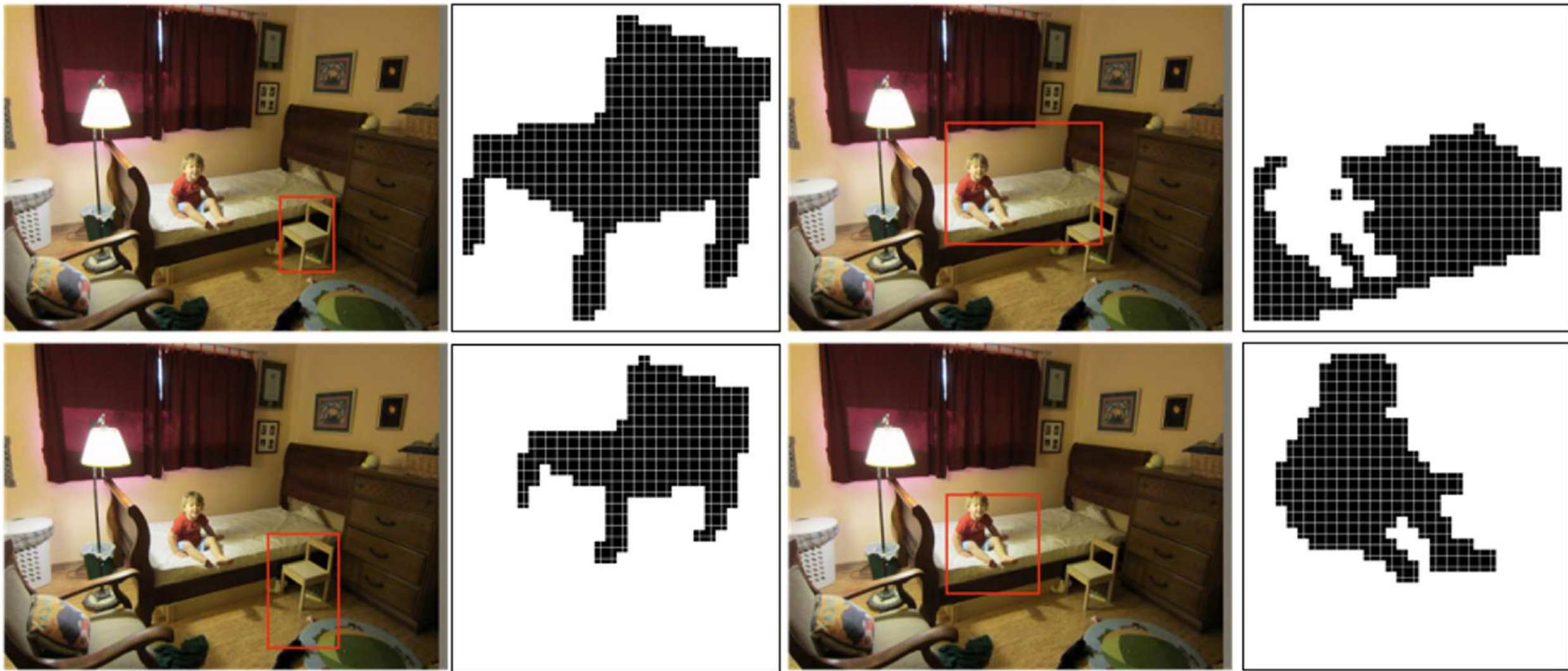


Predict a mask for
each of C classes:
 $C \times 28 \times 28$

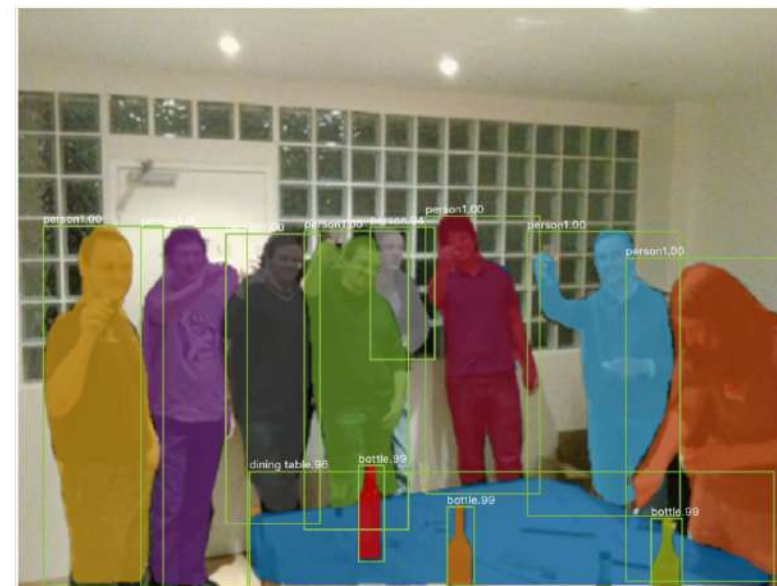
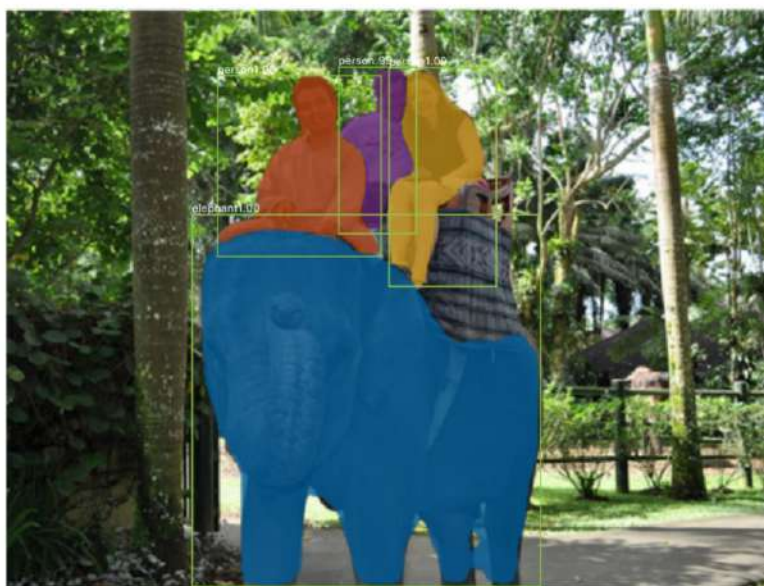
Classification Scores: C

Box coordinates (per class):
 $4 * C$

Mask R-CNN: Example Training Targets



Mask R-CNN: Very Good Results!



Can we generalize Faster R-CNN architecture to other Vision tasks?

Beyond Instance Segmentation: Human Keypoints

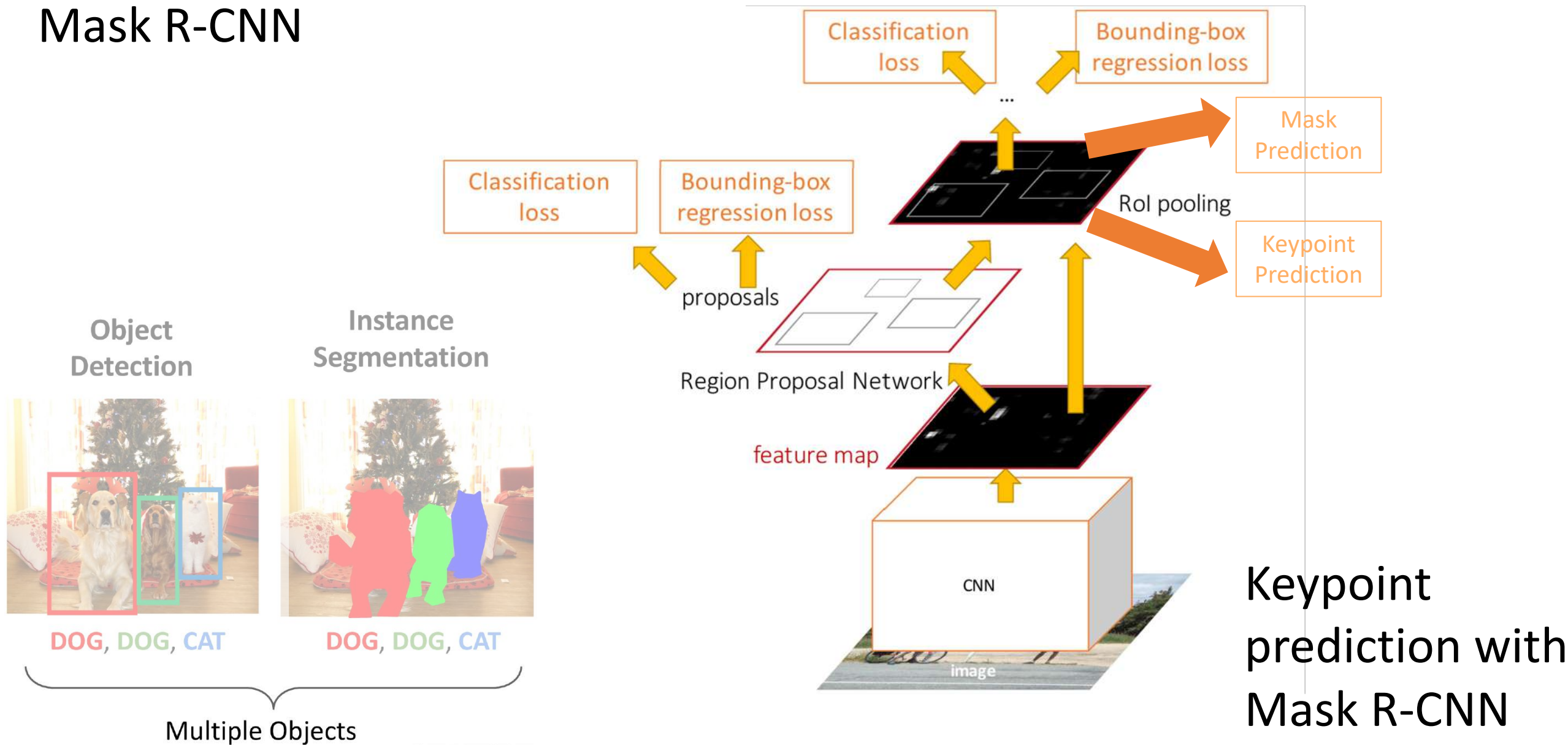
Represent the pose of a human by locating a set of **keypoints**

e.g. 17 keypoints:

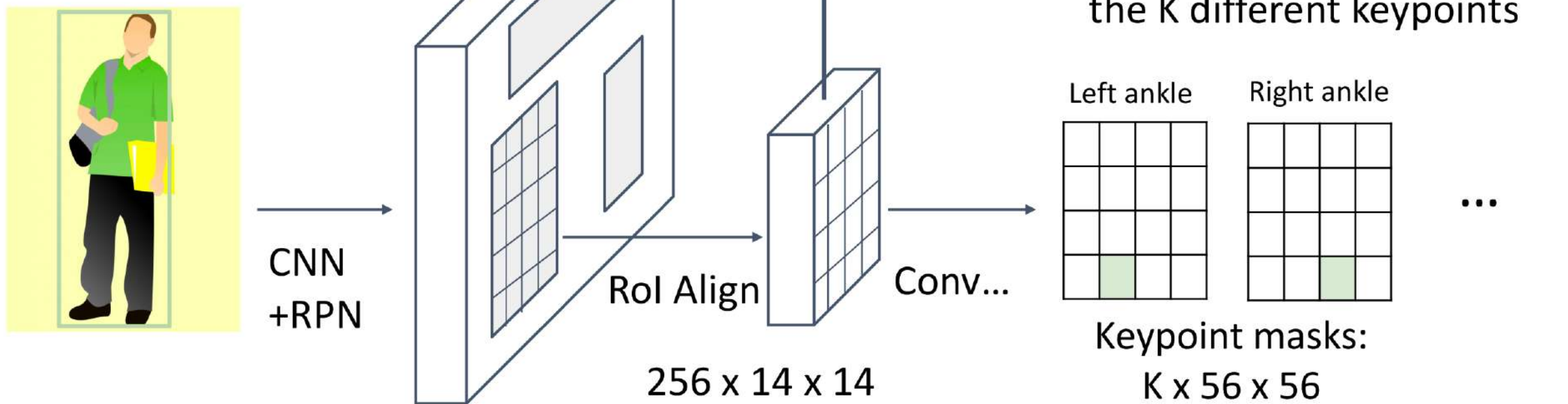
- Nose
- Left / Right eye
- Left / Right ear
- Left / Right shoulder
- Left / Right elbow
- Left / Right wrist
- Left / Right hip
- Left / Right knee
- Left / Right ankle



Instance segmentation with Mask R-CNN



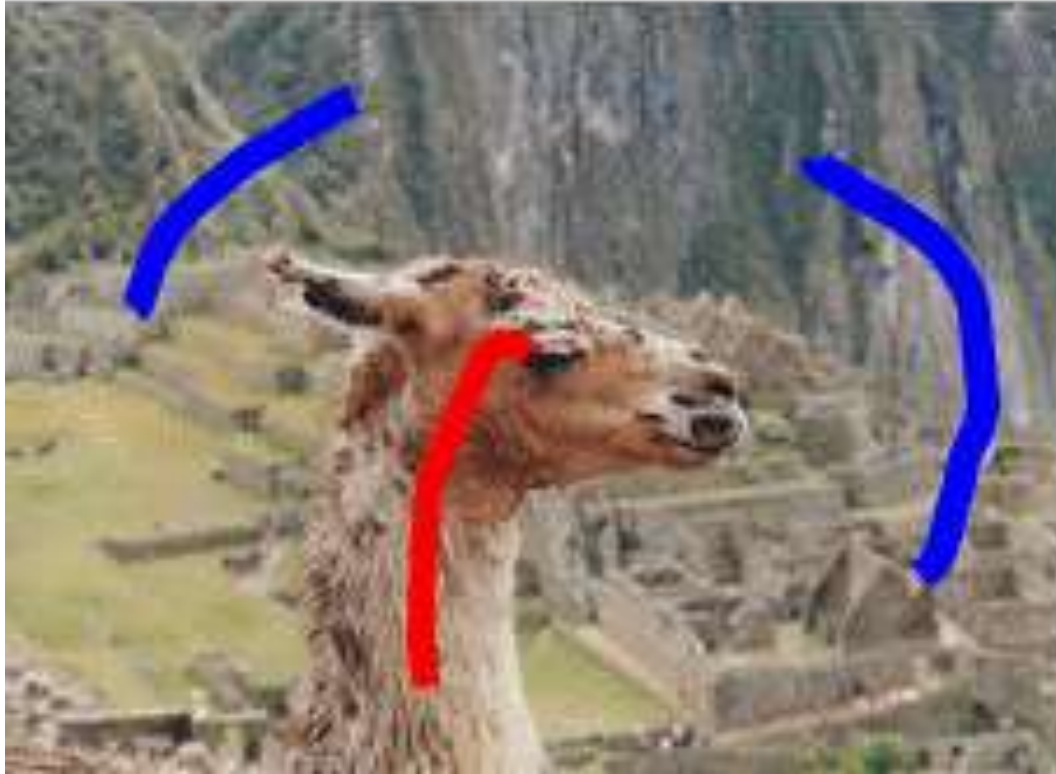
Mask R-CNN: Keypoints



Ground-truth has one “pixel” turned on per keypoint. Train with softmax loss

Interactive Segmentation with Scribbles

(Red = foreground, blue= background)



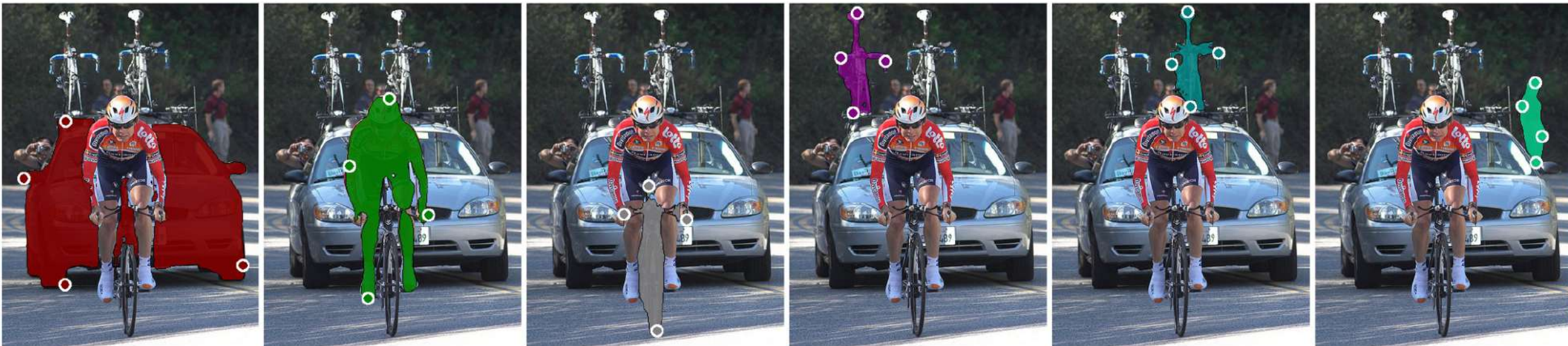
Earlier works of segmentation used Graph Cut techniques to solve this problem.

Normalized cuts and image segmentation

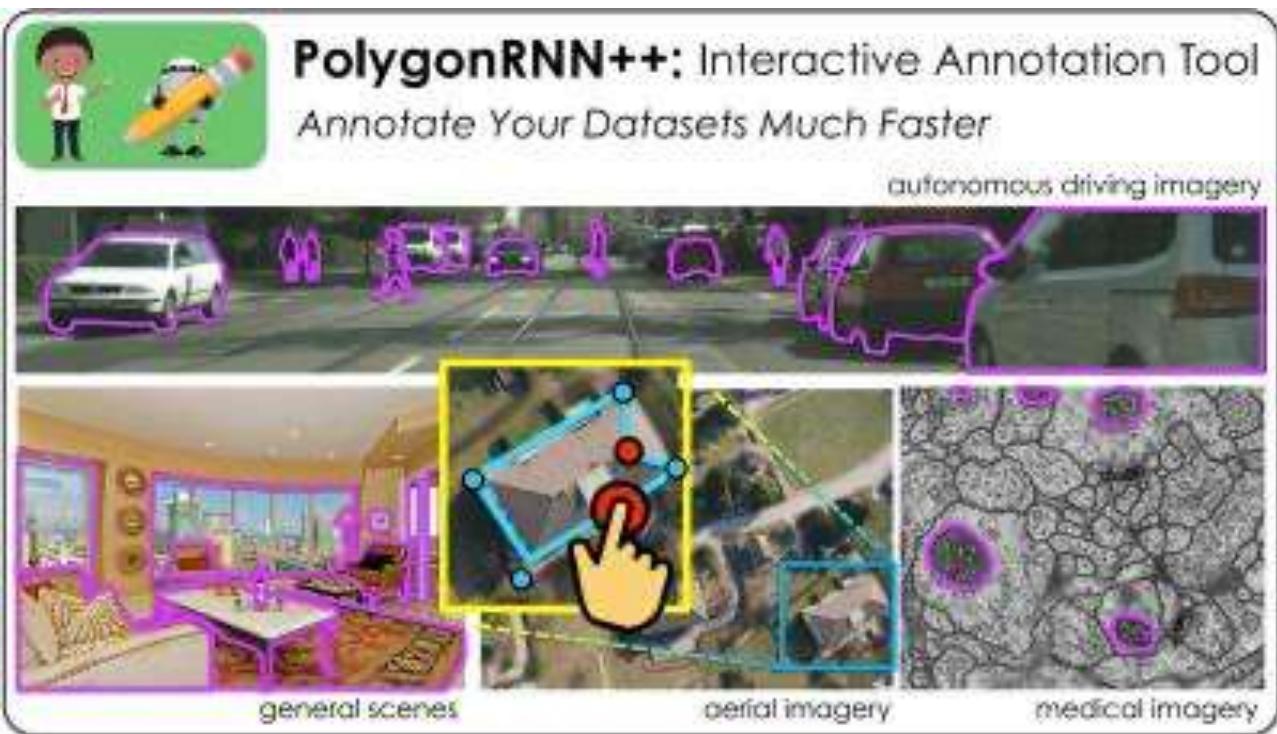
[J Shi, J Malik](#) - IEEE Transactions on pattern analysis and ..., 2000 - ieeexplore.ieee.org

... have smaller **cut** value than the **cut** that partitions ... **cut** cost as a fraction of the total edge connections to all the nodes in the graph. We call this disassociation measure the **normalized cut** (...)

☆ Save ↗ Cite Cited by 19705 Related articles All 34 versions



Deep Extreme Cut (DEXTR): From Extreme Points to Object Segmentation, CVPR 2018

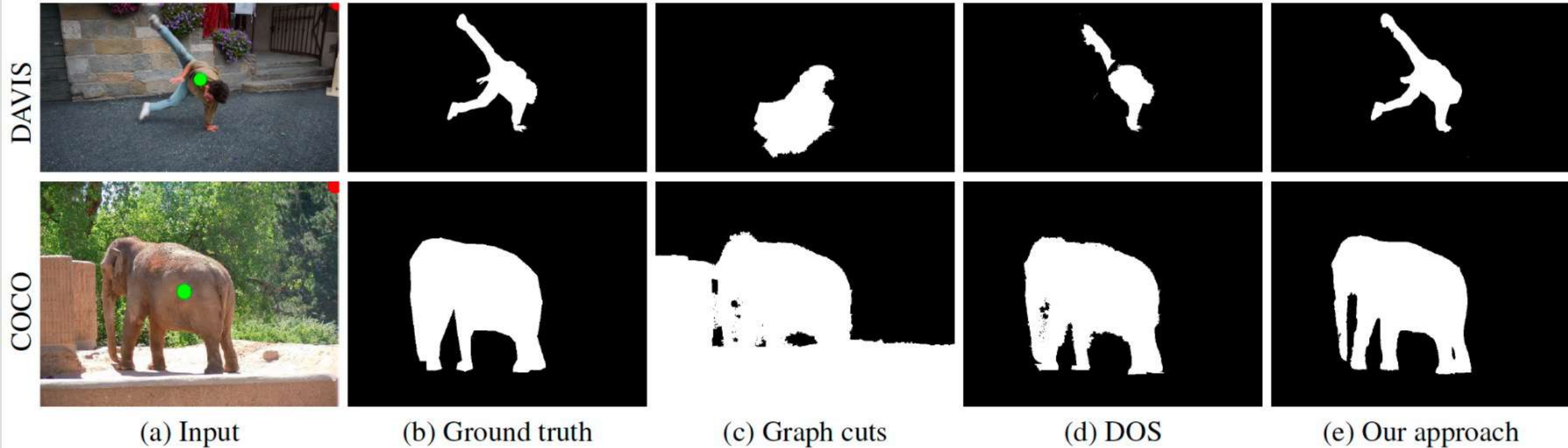


Interactive Segmentation with few points

Efficient Annotation of Segmentation Datasets
with Polygon-RNN++, CVPR 2018

Interactive Segmentation with 2 points

(Green = foreground, red= background)



Interactive Image Segmentation with Latent Diversity, CVPR 2018

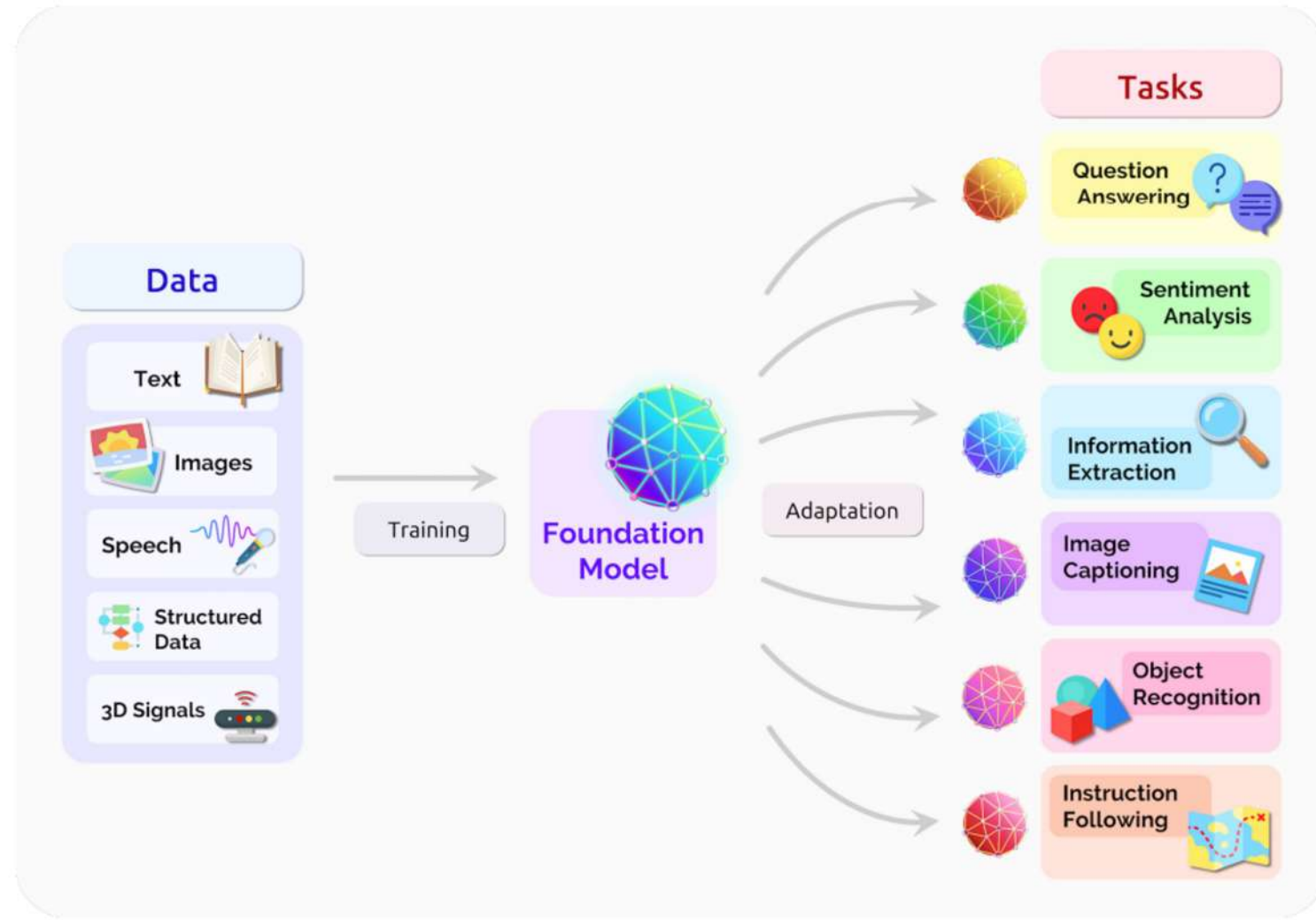
Interactive Segmentation with points



SimpleClick: Interactive Image Segmentation with Simple Vision Transformers, Liu et al. 2022.

Foundation Models and Promptable Segmentation

A foundation model can centralize the information from various modalities.



Segment Anything Model (SAM): the first foundation model for promptable segmentation.



Prompt it with interactive points and boxes



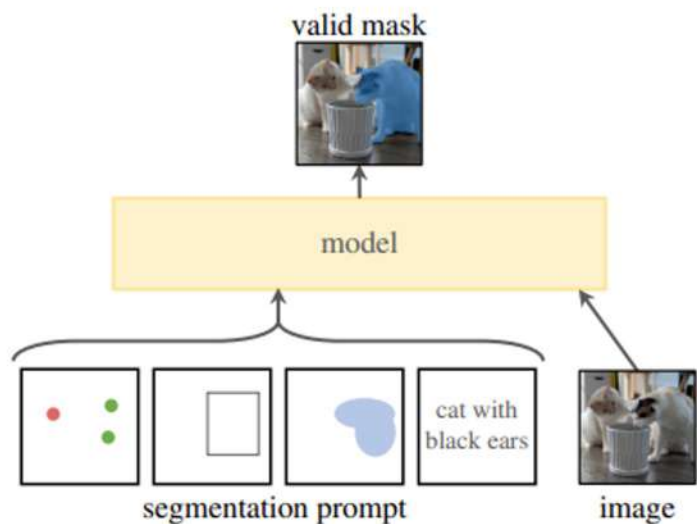
Automatically segment everything in an image



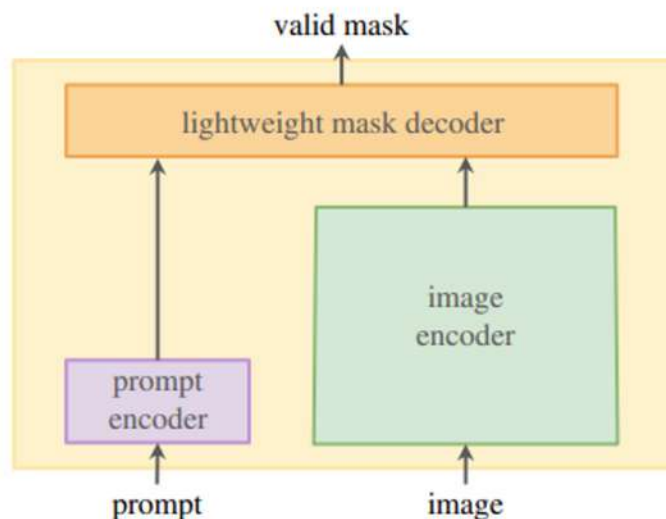
Generate multiple valid masks for ambiguous prompts

Try the demo: <https://segment-anything.com/demo>

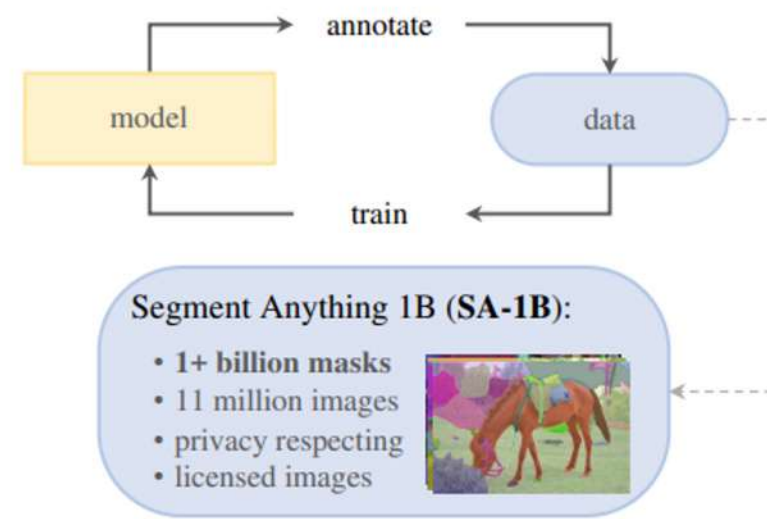
SAM is built with three interconnected components: A task, an model, and a data engine.



(a) **Task:** promptable segmentation

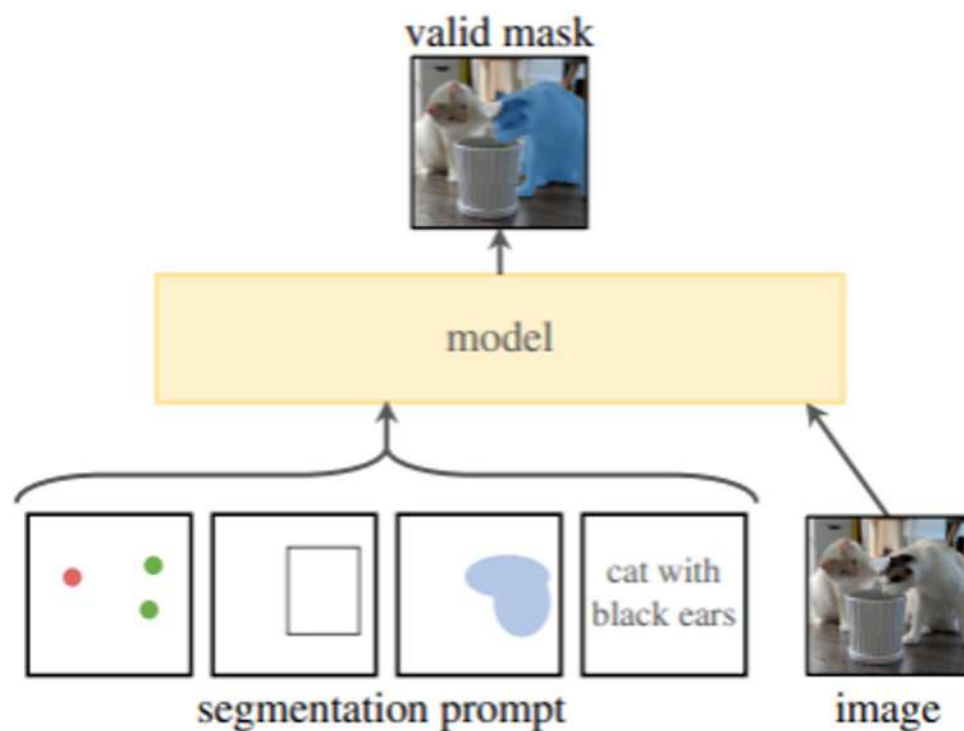


(b) **Model:** Segment Anything Model (SAM)



(c) **Data:** data engine (top) & dataset (bottom)

Task: Promptable Segmentation

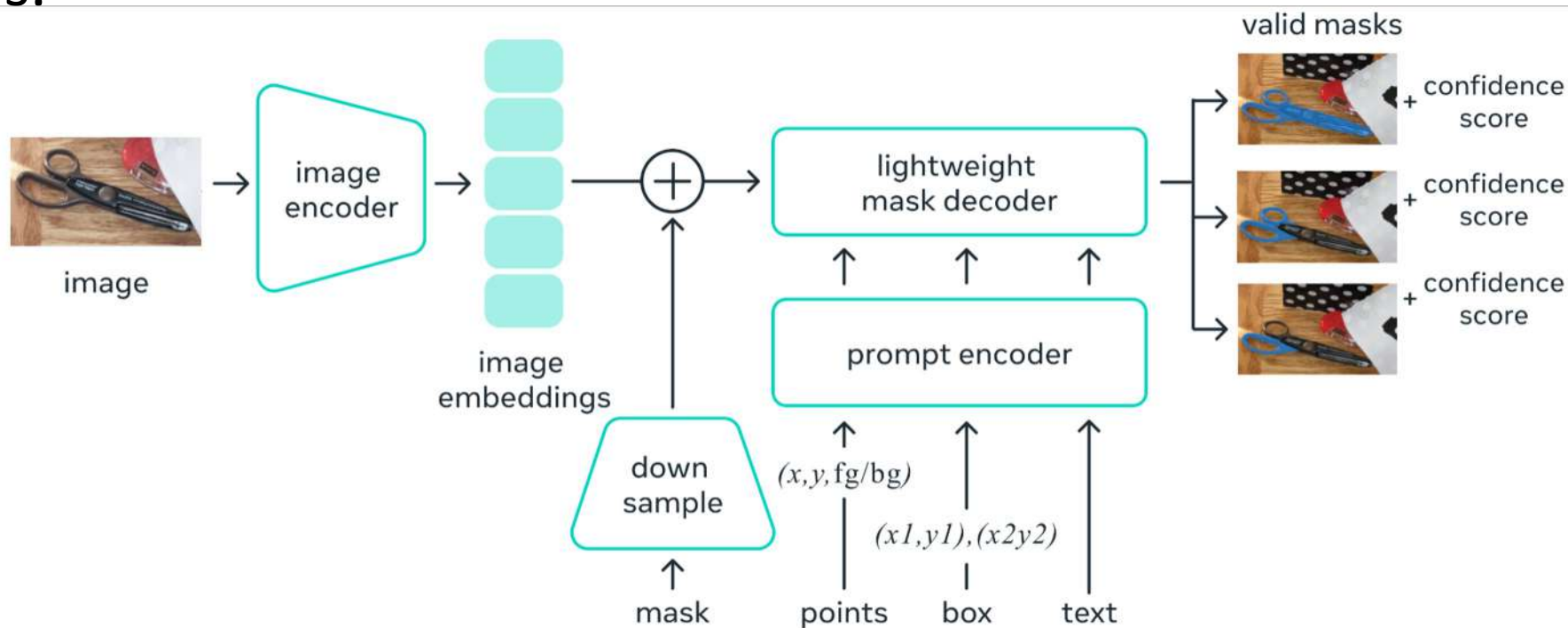


(a) **Task:** promptable segmentation

- SAM considers two sets of prompts: **sparse** (clicks, boxes, text) and **dense** (masks).
- SAM's promptable design enables flexible integration with other systems (i.e., used as **component** in larger systems).

Model: Segment Anything Model (SAM)

- A heavyweight **image encoder** outputs an image embedding.
- A lightweight **prompt encoder** efficiently queries the image embedding.
- A lightweight **mask decoder** produces object masks and confidence scores.

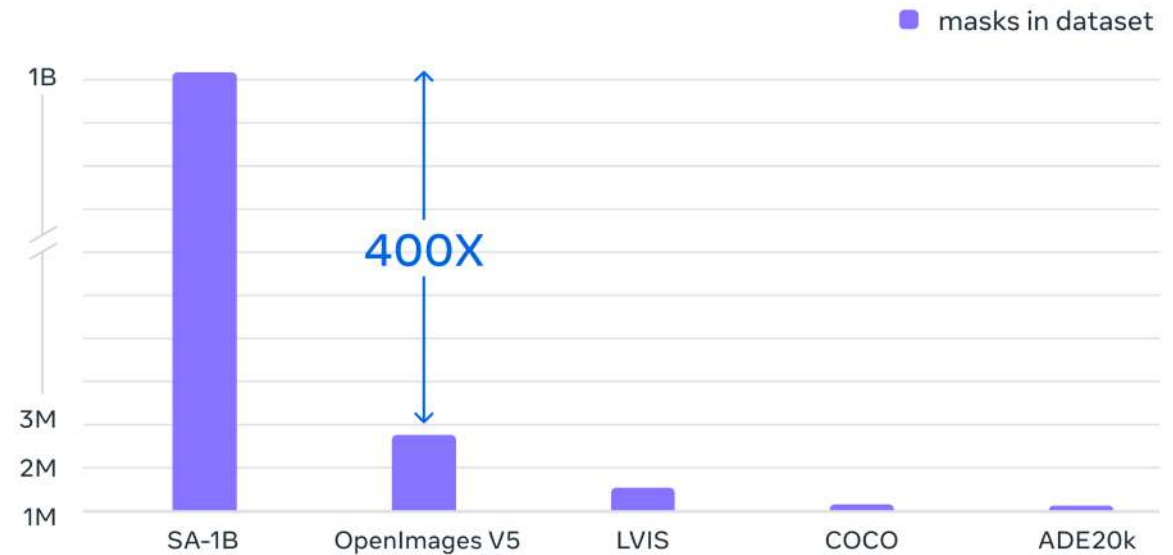
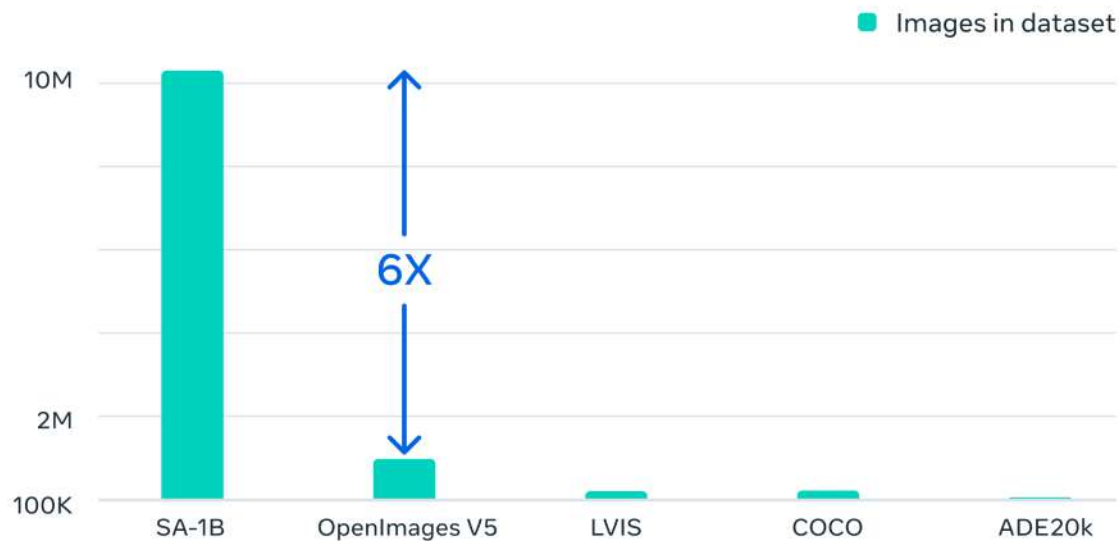


Ambiguity-Aware Segmentation

- SAM is designed to predict multiple masks (i.e., 3 masks: whole, part, subpart) for a single prompt.
- During training, the model only backprops the minimum loss over masks.
- To rank masks, the model predicts a confidence score (i.e., estimated IoU) for each mask.

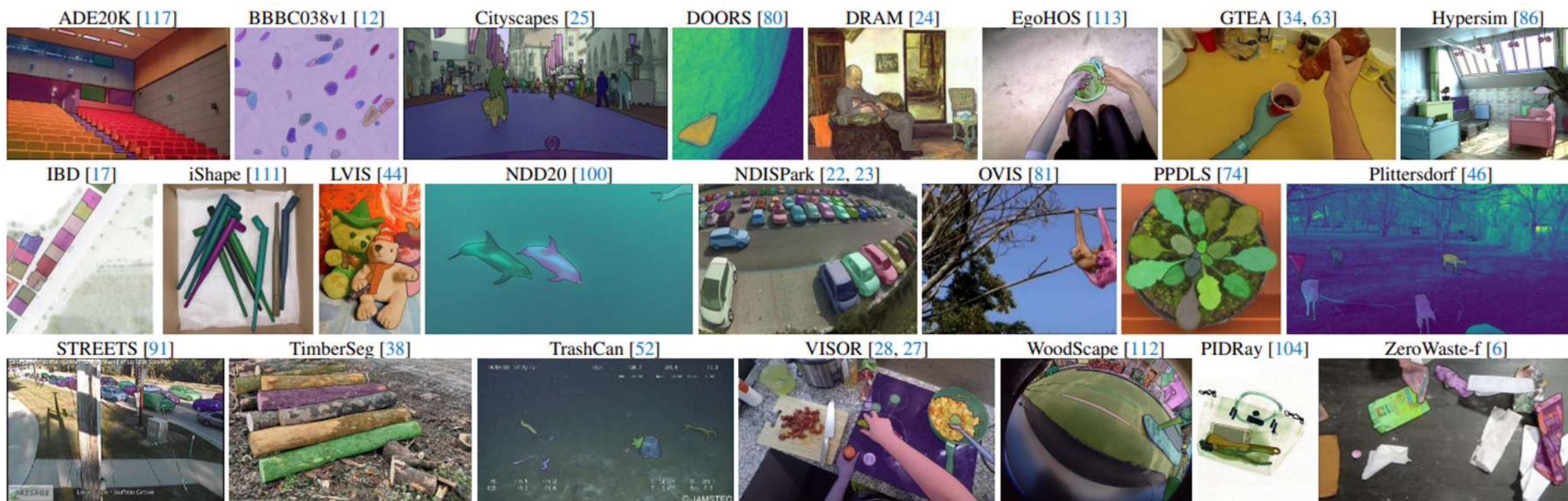
Dataset: SA-1B

- Built with a SAM model in the loop
- 11M images with 1.1B segmentation masks
- 400x more masks than any prior segmentation dataset

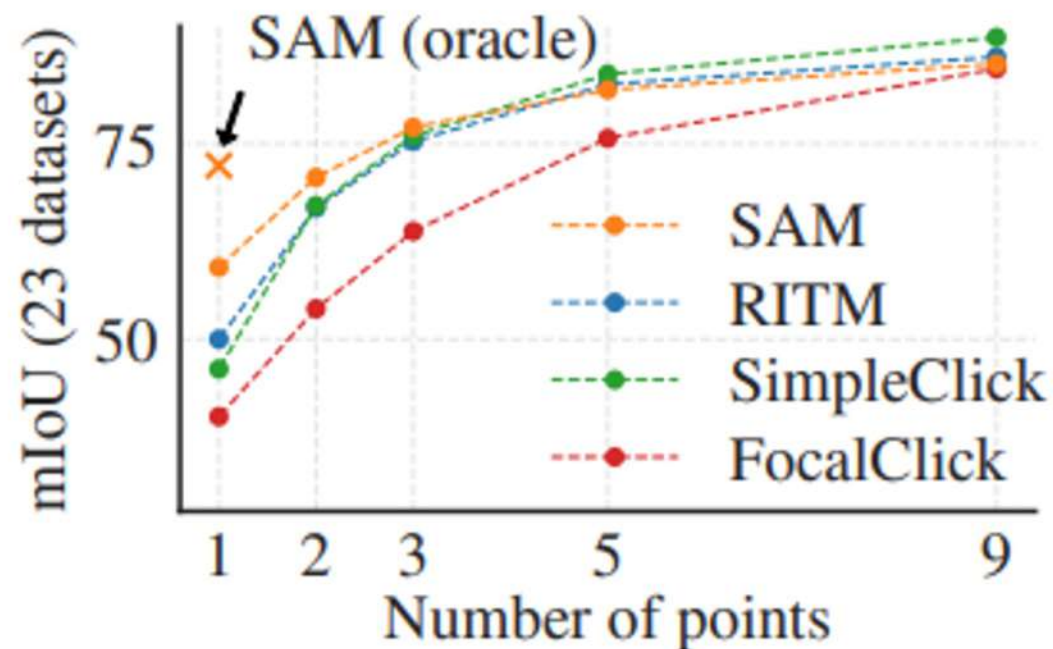


Task 1: Zero-Shot Single Point Valid Mask Evaluation

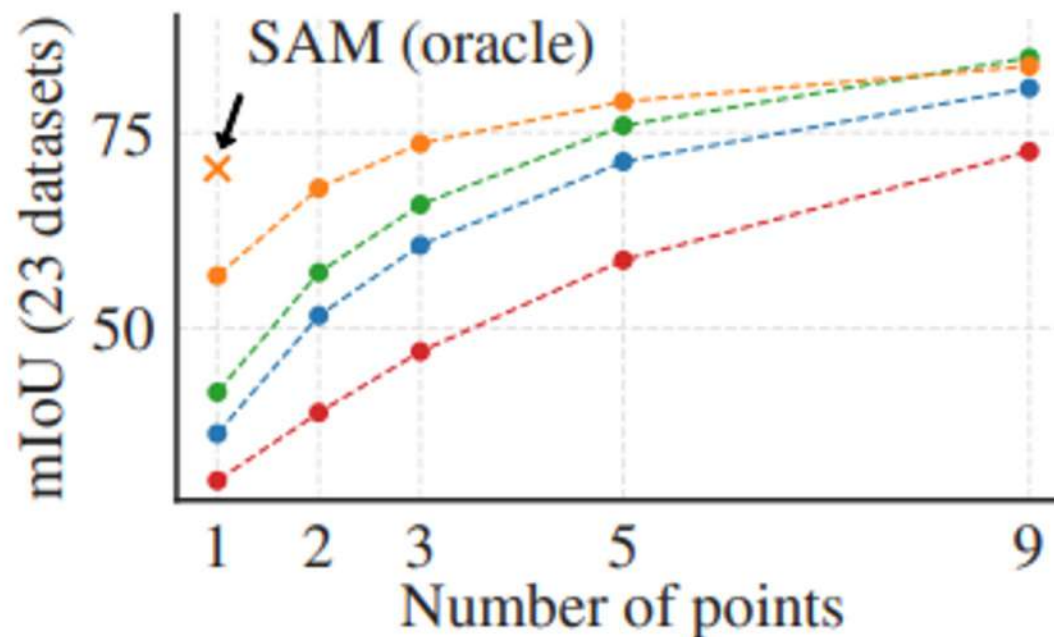
- Training dataset: the whole SA-1B dataset
- Test datasets: 23 diverse segmentation datasets (only validation/test sets??)



Task 1: Zero-Shot Single Point Valid Mask Evaluation



(c) Center points (default)



(d) Random points

SAM significantly outperforms baselines with 1 point and is on par with more points.

Limitations

- SAM may miss fine structures, hallucinate small disconnected components at times, and produce wrong boundaries.
- SAM is expected to be outperformed by dedicated interactive segmentation methods (e.g., SimpleClick) when many points are provided.
- SAM is expected to be outperformed by domain-specific tools (e.g., ilastik).
- SAM's performance on the text-to-mask task is not entirely robust.
- While SAM is initialized with a self-supervised technique (i.e., MAE), the vast majority of its capabilities come from large-scale ***supervised*** training.

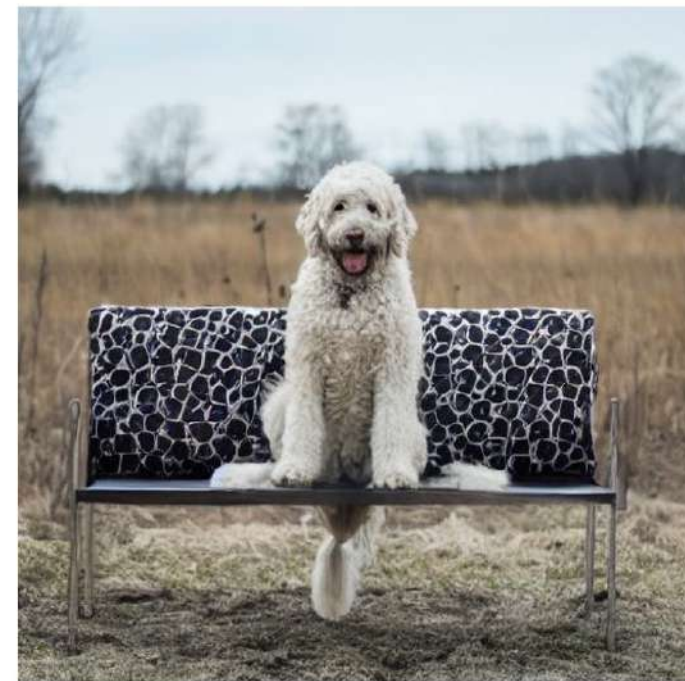
Grounded-Segment-Anything



Text Prompt: Bench



Grounded-SAM Output

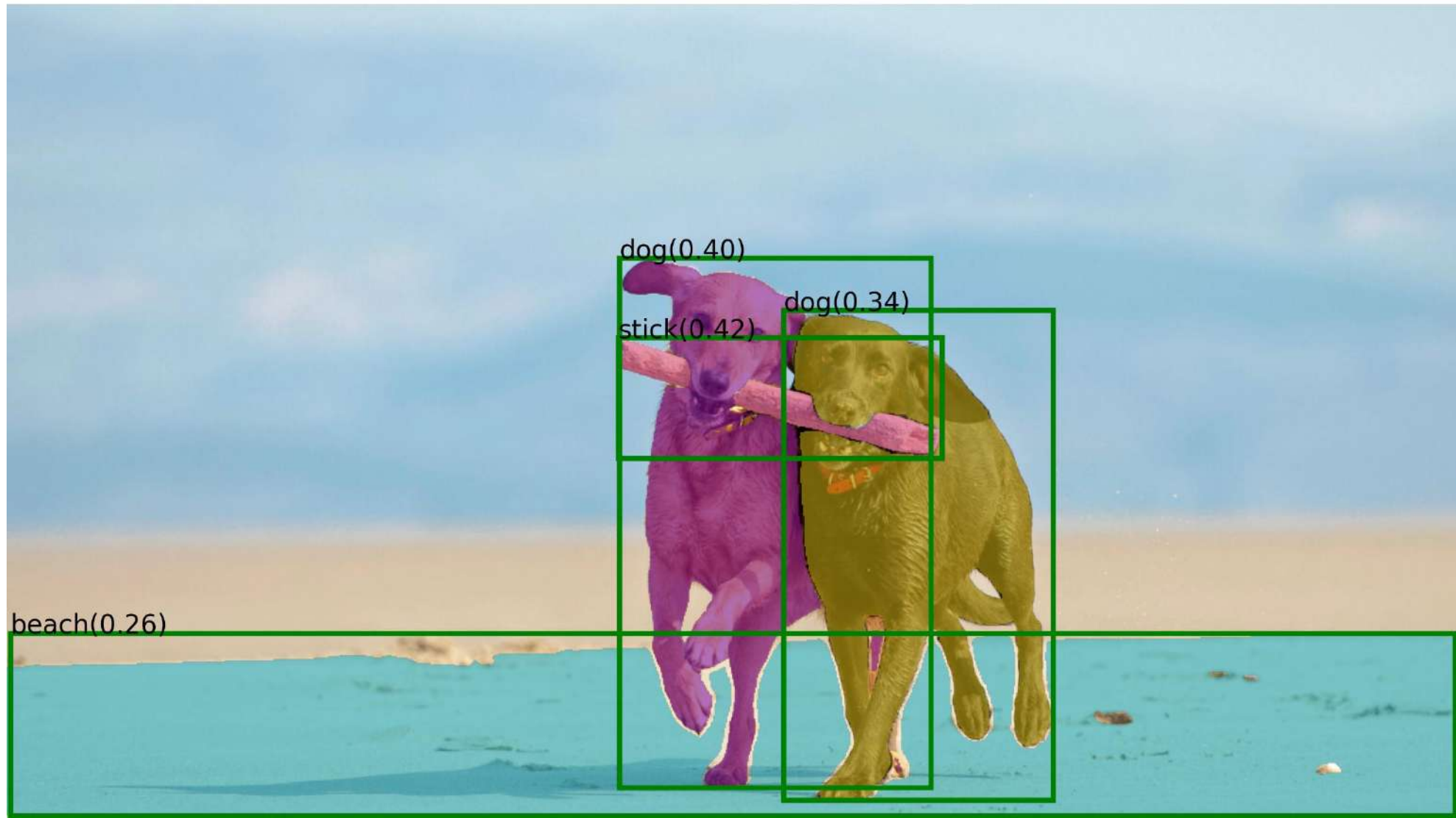


Stable-Diffusion Inpainting
A Sofa, high quality, detailed

BLIP + Grounded-SAM: Automatic Label System!

Using BLIP to generate caption, extract tags and using Grounded-SAM for box and mask generating. Here's the demo output:

there are two dogs playing with a stick on the beach



Video Object Segmentation



DAVIS: Densely Annotated Video Segmentation

Unsupervised: the user does not interact with the algorithm to obtain the segmentation masks. Methods should provide a set of object candidates.

Semi-supervised: user inputs full mask of the object of interest in the first frame only.

Interactive: user gives iterative refinement inputs to the algorithm, in the form of a scribble, to segment the objects of interest.

Beyond Segmentation: Alpha Matting

Image



Alpha matte



Composed Image



Under-constrained!

$$I = \alpha * F + (1 - \alpha) * B$$

Solving Matting with user annotation!

Image



Trimap

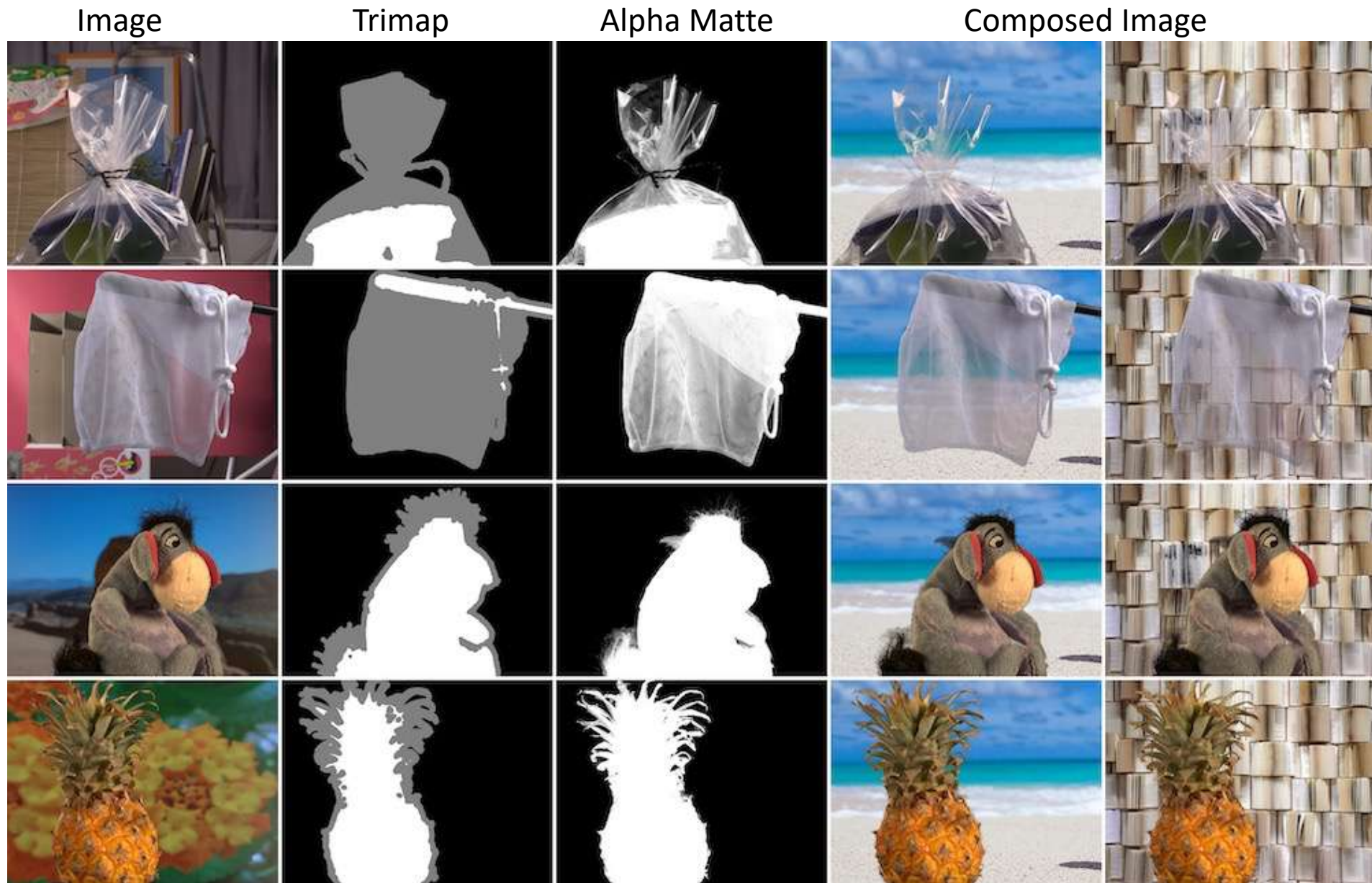


Alpha Matte

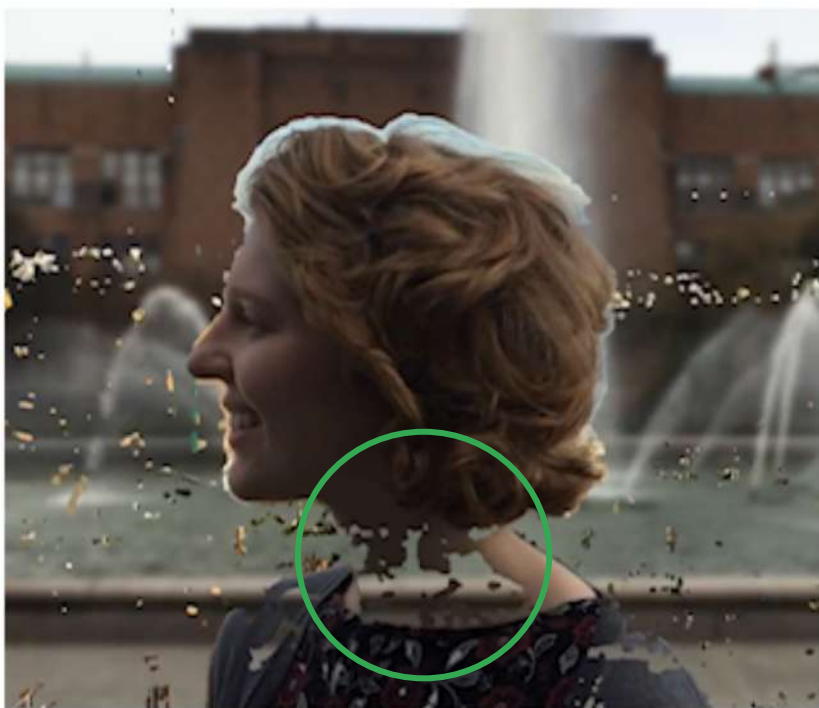


User Annotation during inference

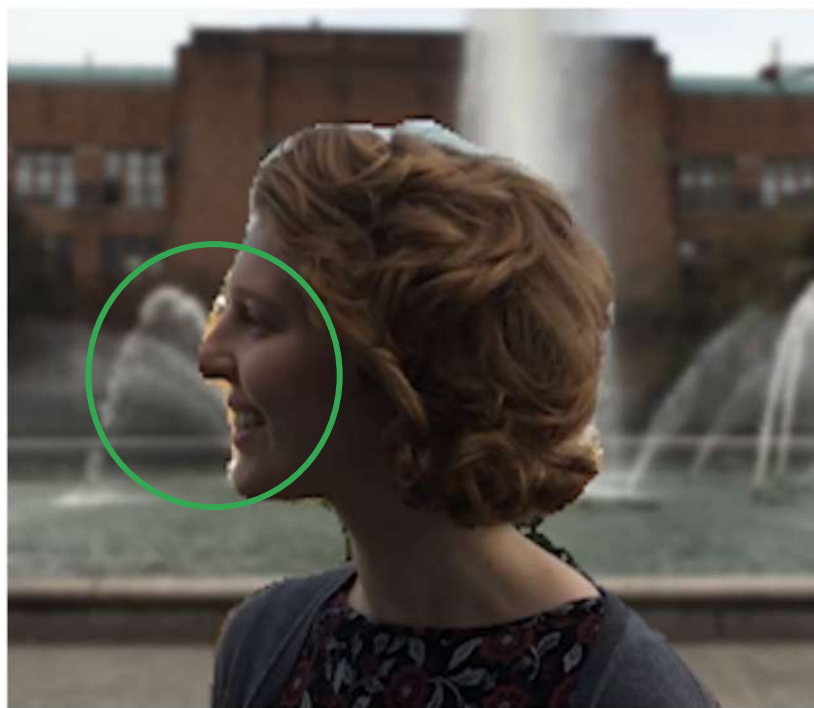
Image Matting with Trimap



Background Subtraction



Segmentation



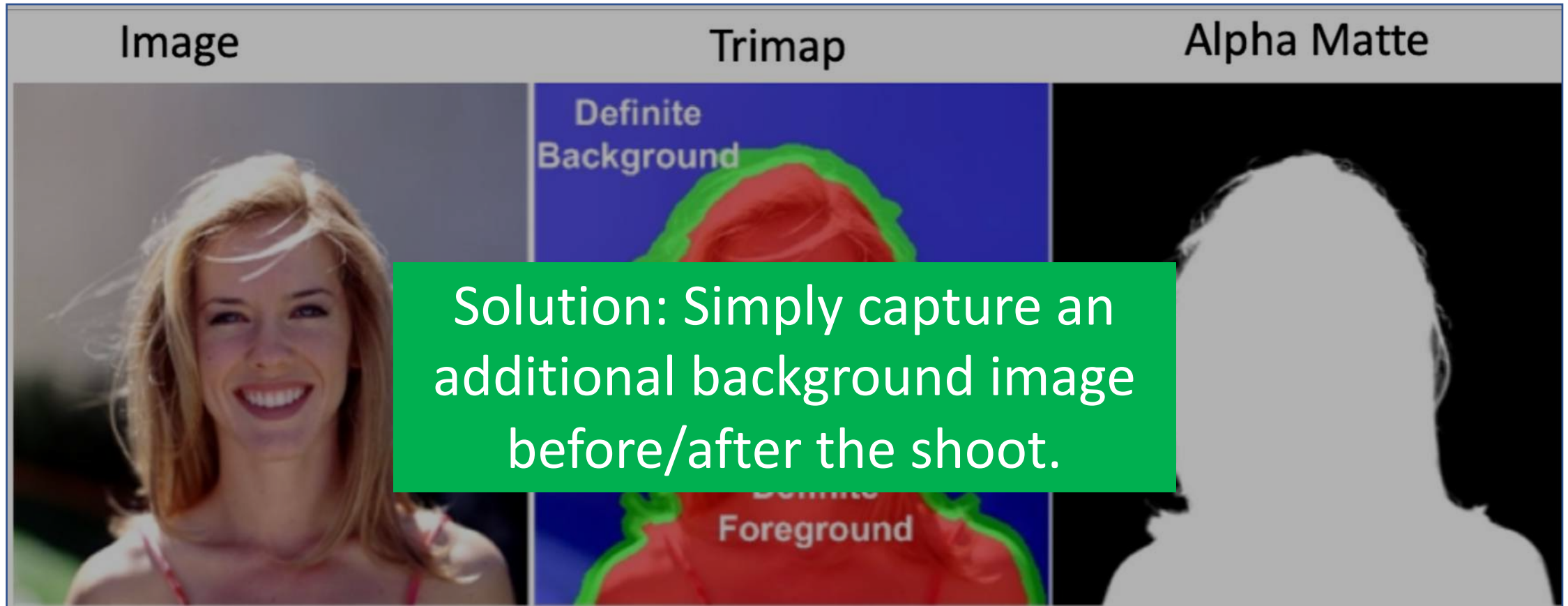
Vs

Matting



- Binary Mask
- No color separation

Solving Matting with user annotation!



Solution: Simply capture an additional background image before/after the shoot.



User Annotation during inference

Expensive – especially for a video

Background Matting: The World is Your Green Screen

Soumyadip Sengupta

Vivek Jayaram

Brian Curless

Steve Seitz

Ira Kemelmacher-Shlizerman

University of Washington



Original Video



Replaced Background

Challenges: Lack of High-Quality Training Data

Instances from Adobe Matting Dataset (Ground Truth)

- ✓ **Very High Quality**
Manually extracted alpha and foreground
- ✗ **Very Few Training Samples**
Only 500+ samples in total



Alpha



Foreground

Technical solutions:

- Make the problem easy with user interaction
- Create realistic synthetic data
- Unsupervised Finetuning on real data.

Adobe Matting Dataset

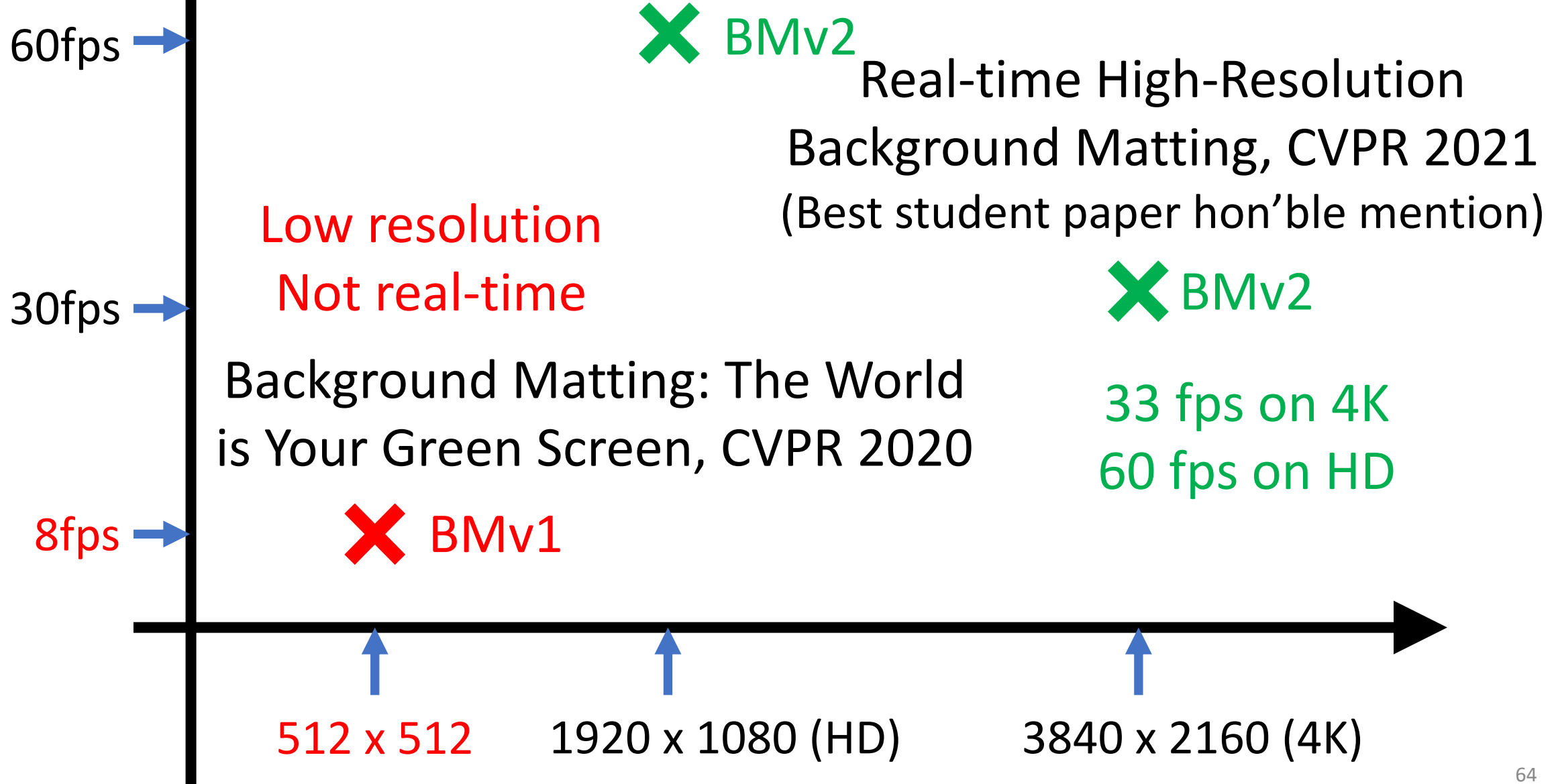
+

Distinctions-646 Dataset

ML models trained on this data alone,
fails to generalize.



Speed and Resolution



Measured on Nvidia RTX 2080 Ti with batch size 1



Mute



Start Video



Security



Participants 1



Share Screen



Record



Breakout Rooms

65

End

Observation: Only few regions are transparent.

Solution:

- Predict alpha matte at low resolution
- Only refine transparent regions at high-res.

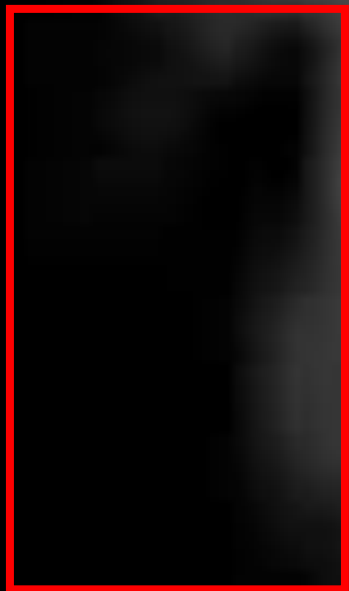


How do we find transparent regions to refine at high-res?



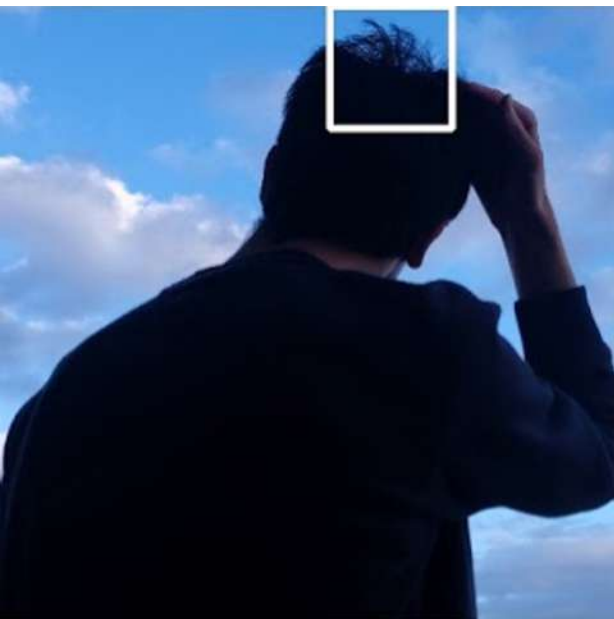








Input



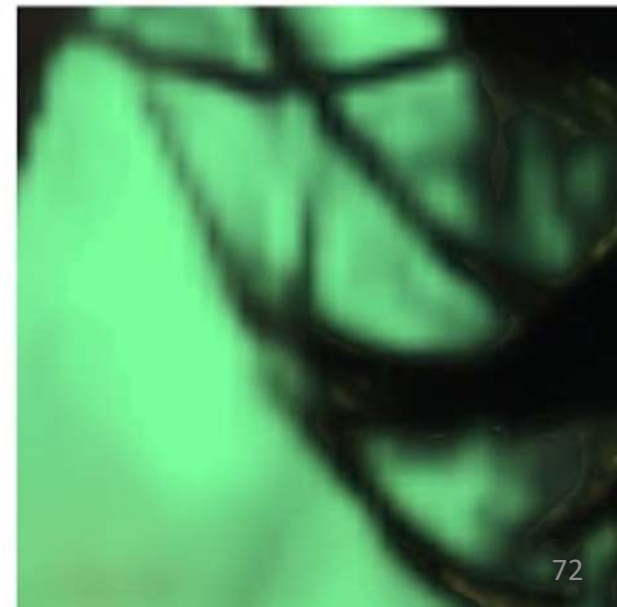
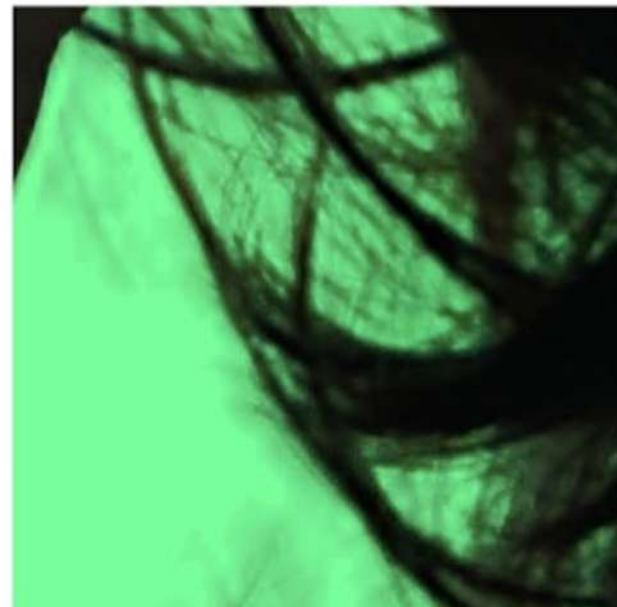
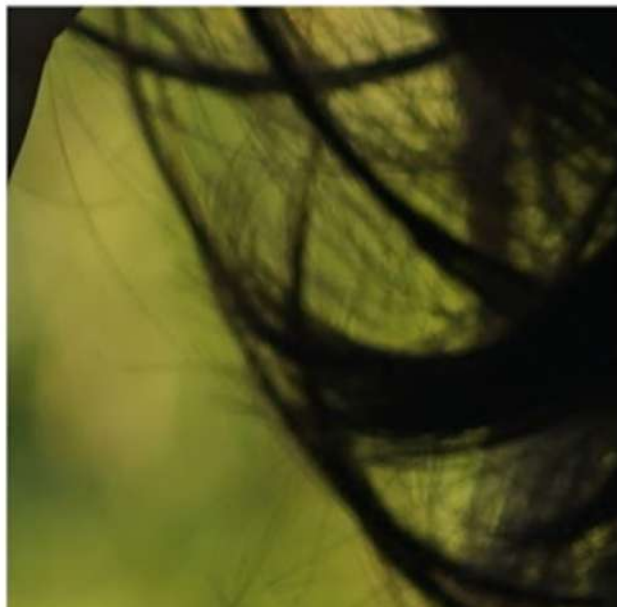
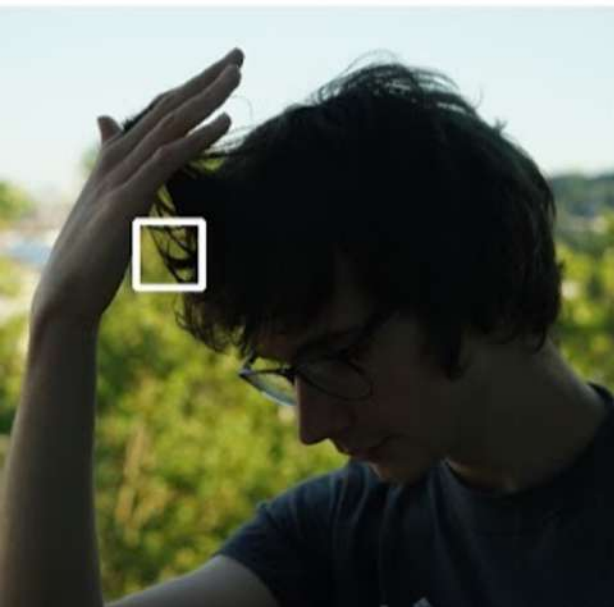
Input (zoomed)



BMv2 [CVPR 2021]



BMv1 [CVPR 2020]



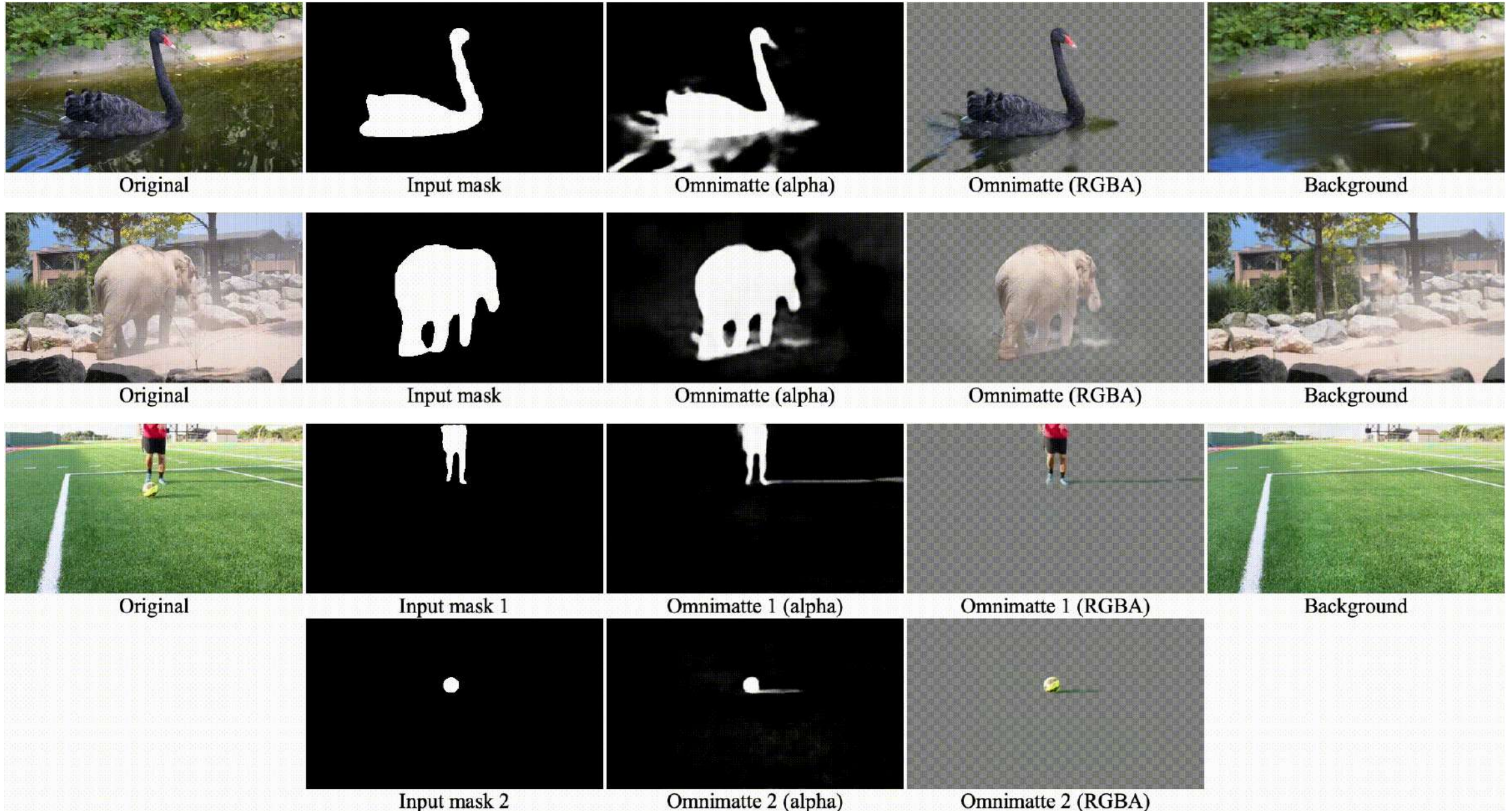
Drawbacks of Background Matting:

X requires explicit background capture.

X can't handle large camera motion.

Idea: Use RNN to aggregate temporal motion information to separate foreground and background
Robust Video Matting, WACV 2022.

Alpha Matting in presence of foreground-background interaction



Input



Input



Recap

- Semantic segmentation: detect masks of specific object classes
- Instance segmentation: detect masks of each instances of specific object class
- Panoptic segmentation: Semantic + Instance combined.

Evolution of Neural network architectures to solve segmentation:

- Fully convolutional networks (VGGlike)
- Mask R-CNN (instance segmentation)
- U-Net
- Transformer based architecture (Swin V2)

Interactive Segmentation (scribbles, points, text prompts etc):

- Segment Anything (SAM)
- Way more robust than semantic/instance segmentation

Beyond Image segmentation:

- Video object segmentation (unsupervised, weak-supervised, interactive)
- Alpha Matting

Slide Credits

- EECS 442/498 [Computer Vision](#), by Justin Johnson & David Fouhey, U Michigan.